

Surrogate Modeling of the Rosenbrock and Trid Functions: Construction, Validation, and Limits of Trust

Bahaa Karawia*

King Fahd University of Petroleum and Minerals, Dhahran 31261, Saudi Arabia

AE 541 Digital Twin in Modern Engineering • Assignment 1

Bahaa Karawia, KFUPM ID 202510630 • Assigned function: Trid (2 variables)

Code, results and report sources: <https://github.com/BahaaIbrahimK/AE541-Assignment1-Surrogate-Modeling>

Two benchmark functions are treated as expensive simulations to test how far a cheap surrogate can be trusted. For the Rosenbrock function (common) and the two-variable Trid function (assigned), 60 evaluations from a maximin Latin hypercube design are split into 45 training and 15 held-out test points. Three surrogates are compared: polynomial regression with its degree chosen by five-fold cross-validation on the training set only, a cubic radial basis function (RBF) interpolant, and a Gaussian process (GP). They are assessed with test metrics, side-by-side error maps, learning curves, surrogate-based optimization with verification, and a reduced-box extrapolation test. Cross-validation recovers the true polynomial degrees (4 and 2), and the fitted coefficients match the exact expansions to within 4×10^{-13} . The GP reaches a test normalized root-mean-square error of 7.8×10^{-6} and 1.0×10^{-6} and places the two minima within 0.013 and 6×10^{-4} of the truth. The RBF fails at the domain boundaries and creates a spurious negative minimum in the Rosenbrock valley, which only verification exposes. Outside a reduced training box, errors grow by factors of up to 2300. The hand-derived Trid minimum, $f = -2$ at $(2, 2)$, is confirmed analytically and numerically.

Nomenclature

$\mathbf{A}, \mathbf{P}$	=	RBF kernel matrix and linear-tail matrix
d	=	distance from a surrogate minimum to the known minimum
$f, \hat{f}$	=	true function and surrogate prediction
$\mathcal{F}_k$	=	k -th cross-validation fold
$\mathbf{H}$	=	Hessian matrix
$k(\cdot, \cdot), \mathbf{K}, \mathbf{k}_*$	=	GP kernel, training covariance matrix, and training–test covariances

*PhD Student, Aerospace Engineering Department; KFUPM ID 202510630. AE 541 Digital Twin in Modern Engineering, Assignment 1; assigned function (Part B): Trid function, 2 variables.

ℓ_j	=	GP length scale of input j
N, n, n_v	=	main DOE budget, number of training points, number of validation points
n_{terms}	=	number of polynomial coefficients
p	=	total degree of the polynomial surrogate
R^2	=	coefficient of determination
μ_j, s_j	=	training mean and standard deviation of input j (scaling)
$\mathbf{u}$	=	design point in unit-square coordinates
$\mathbf{x}, \mathbf{z}$	=	physical and standardized inputs
$\mathbf{x}^*$	=	location of the global minimum
β	=	polynomial coefficients
ε_{gen}	=	generalization error
$\hat{\mu}, \hat{\sigma}$	=	GP posterior mean and standard deviation
π_j	=	random permutation used by the Latin hypercube
σ_f^2, σ_n^2	=	GP signal and noise variances
$\phi_{\min}$	=	minimum pairwise distance (maximin criterion)
ψ, Ψ	=	vector of monomials and polynomial design matrix
w_i, c_j	=	RBF weights and linear-tail coefficients
CV	=	cross-validation
DOE	=	design of experiments
GP	=	Gaussian process
LHS	=	Latin hypercube sampling
MAE, MaxAE	=	mean and maximum absolute error
(N)RMSE	=	(normalized) root-mean-square error
RBF	=	radial basis function

I. Introduction

High-fidelity analyses such as CFD, finite-element models and coupled aero-structural solvers are too expensive to call thousands of times inside a design loop. The usual answer is a *surrogate model*: a cheap approximation fitted to a small, deliberately chosen set of expensive runs, which then stands in for the analysis during exploration and optimization [1, 2]. A surrogate is useful only if the engineer can say how wrong it is and where. The question behind this study is therefore not “how low can the error go” but “can a cheap model guide a design decision reliably, and inside which bounds” [3].

Two known two-variable functions are treated as if they were expensive simulations. Part A uses the Rosenbrock function [4], common to the whole class; Part B uses the two-variable Trid function [5], assigned to the author. Because both functions are known everywhere, every claim a surrogate makes can be checked against the truth, a luxury that a real project never has. For each function the complete workflow of the course reference notebook [6] is applied: definition of the function and domain, design of experiments (DOE), a leakage-free train/test split, three surrogate types, complexity selection by training-only cross-validation (CV), held-out validation, side-by-side error maps, uncertainty quantification, learning curves, surrogate-based optimization with verification, and an extrapolation test. The shared theory is given once in Sec. II; results for the two functions follow in Secs. III and IV, and Sec. V compares them and states where each model can be trusted.

II. Methodology

A. Problem Statement

Let the expensive model be a deterministic map $f : \mathcal{X} \subset \mathbb{R}^2 \rightarrow \mathbb{R}$ on the box $\mathcal{X} = [l_1, u_1] \times [l_2, u_2]$. From a design $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ with $y^{(i)} = f(\mathbf{x}^{(i)})$, a surrogate $\hat{f}(\mathbf{x}; \boldsymbol{\theta}) \approx f(\mathbf{x})$ is built. The quantity of interest is the generalization error over the whole domain,

$$\varepsilon_{\text{gen}}^2 = \frac{1}{|\mathcal{X}|} \int_{\mathcal{X}} (f(\mathbf{x}) - \hat{f}(\mathbf{x}))^2 d\mathbf{x}. \quad (1)$$

Equation (1) needs f everywhere, so in practice it can only be *estimated* from data the model has not seen (a held-out test set, or CV). Here it can also be evaluated on a dense 120×120 grid, which is used purely as a diagnostic.

The two functions are listed in Table 1. Both are polynomials: expanding the squares gives

$$f_{\text{R}}(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2 = 100x_1^4 - 200x_1^2x_2 + 100x_2^2 + x_1^2 - 2x_1 + 1, \quad (2)$$

$$f_{\text{T}}(x_1, x_2) = (x_1 - 1)^2 + (x_2 - 1)^2 - x_1x_2 = x_1^2 + x_2^2 - x_1x_2 - 2x_1 - 2x_2 + 2, \quad (3)$$

so the Rosenbrock function is exactly of total degree 4 and the Trid function exactly of degree 2. This fact is known here but is *not* used to choose any model setting; Sec. III.D shows that CV recovers it from the data alone.

Table 1 Test functions, domains, known global minima and the exact range of f on each box.

Function	Domain	$\mathbf{x}^*$	$f(\mathbf{x}^*)$	Range of f	Exact degree
Rosenbrock, Eq. (2)	$[-2, 2]^2$	(1, 1)	0	0 to 3609	4
Trid, Eq. (3)	$[-4, 4]^2$	(2, 2)	-2	-2 to 50	2

B. Design of Experiments

The main budget is $N = 60$ evaluations, counting training and test points together. All designs are generated in the unit square and mapped linearly to the physical box, so their space-filling quality can be compared independently of the domain size. Three designs are compared at the same N : uniform random sampling; a Latin hypercube (LHS), which splits each axis into N bins and places exactly one point in each bin [7],

$$u_j^{(i)} = \frac{\pi_j(i) - 1 + U_{ij}}{N}, \quad U_{ij} \sim \mathcal{U}(0, 1), \quad (4)$$

where π_j is a random permutation of $\{1, \dots, N\}$; and a *maximin LHS*, the best of 100 independent LHS draws under the maximin criterion [8]

$$\phi_{\min} = \min_{i \neq k} \|\mathbf{u}^{(i)} - \mathbf{u}^{(k)}\|_2. \quad (5)$$

Generating candidate designs costs no function evaluations, so screening them improves the spread for free while keeping the one-dimensional stratification of the LHS. Equations (4) and (5) are written in unit-square coordinates, so $\phi_{\min}$ can be compared between the two boxes. To avoid judging the designs on a single lucky draw, $\phi_{\min}$ is also computed for 500 independent random and LHS realizations.

C. Train/Test Split and Data Leakage

The 60 samples are split once, at random (seed 7), into 45 training and 15 test points. The test points are set aside immediately and are used only after every modeling choice has been fixed. *Data leakage* means that information from the test data reaches the fitting process, for example when the input scaling (μ_j, s_j) is computed from all 60 points, or when a degree or kernel bound is chosen by looking at test error; either makes the reported error optimistic. It is avoided here in three ways: (i) the standardization $z_j = (x_j - \mu_j)/s_j$ sits inside a scikit-learn pipeline [9], so every fit, including each CV fold, learns μ_j and s_j from its own training rows only; (ii) the polynomial degree and the GP bound study (Sec. II.D) are judged by training-only CV; (iii) test and dense-grid errors are reported as diagnostics and never fed back.

D. Surrogate Models

Polynomial regression (required). A total-degree- p polynomial in the standardized inputs,

$$\hat{f}(\mathbf{x}) = \sum_{a+b \leq p} \beta_{ab} z_1^a z_2^b = \boldsymbol{\psi}(\mathbf{z})^\top \boldsymbol{\beta}, \quad n_{\text{terms}} = \frac{(p+1)(p+2)}{2}, \quad (6)$$

is fitted by least squares, $\boldsymbol{\beta}^* = \arg \min \|\mathbf{y} - \boldsymbol{\Psi} \boldsymbol{\beta}\|_2^2$, where the rows of the design matrix $\boldsymbol{\Psi}$ are the monomials $\boldsymbol{\psi}(\mathbf{z}^{(i)})^\top$. The coefficients β_{ab} are the only parameters. The system is solved by an SVD-based least-squares routine; when n_{terms} exceeds the number of points, the problem is under-determined and this routine returns the minimum-norm solution,

one of infinitely many exact fits. Because an affine input scaling preserves the degree, the fitted model is also an exact degree- p polynomial in the physical $\mathbf{x}$; its physical coefficients are recovered by projecting $\hat{f}$ onto the raw monomials on the grid, which allows a direct comparison with Eqs. (2) and (3).

Cubic radial basis function (RBF) interpolation. The RBF model is a weighted sum of radially symmetric kernels centered on the n training points, plus a linear tail [10]:

$$\hat{f}(\mathbf{z}) = \sum_{i=1}^n w_i \|\mathbf{z} - \mathbf{z}^{(i)}\|^3 + c_0 + c_1 z_1 + c_2 z_2, \quad \sum_i w_i = 0, \quad \sum_i w_i \mathbf{z}^{(i)} = \mathbf{0}. \quad (7)$$

The side conditions in Eq. (7) stop the weights from carrying a linear trend of their own. Imposing $\hat{f}(\mathbf{z}^{(i)}) = y^{(i)}$ gives the symmetric saddle-point system

$$\begin{bmatrix} \mathbf{A} & \mathbf{P} \\ \mathbf{P}^\top & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ \mathbf{c} \end{bmatrix} = \begin{bmatrix} \mathbf{y} \\ \mathbf{0} \end{bmatrix}, \quad A_{ik} = \|\mathbf{z}^{(i)} - \mathbf{z}^{(k)}\|^3, \quad \mathbf{P} = [\mathbf{1}, \mathbf{z}_1, \mathbf{z}_2]. \quad (8)$$

The weights w_i and tail coefficients c_0, c_1, c_2 , obtained by solving Eq. (8), are the parameters; the cubic kernel has no shape parameter to tune. The model interpolates the data exactly, which is appropriate for deterministic simulations, but it reproduces exactly only polynomials of degree at most one (the tail).

Gaussian process (GP) regression. The outputs are modeled as a GP with a squared-exponential kernel with automatic relevance determination (ARD) and a small noise term [11]:

$$k(\mathbf{z}, \mathbf{z}') = \sigma_f^2 \exp\left(-\frac{1}{2} \sum_{j=1}^2 \frac{(z_j - z'_j)^2}{\ell_j^2}\right) + \sigma_n^2 \delta_{\mathbf{z}\mathbf{z}'}. \quad (9)$$

In Eq. (9), σ_f^2 is the signal variance (how far the normalized output swings), ℓ_j is the length scale of input j (how far one must move in z_j before the output changes appreciably, so a small ℓ_j marks a sensitive direction), and σ_n^2 absorbs numerical noise. With $\mathbf{K} = k(\mathbf{Z}, \mathbf{Z})$ and $\mathbf{k}_* = k(\mathbf{Z}, \mathbf{z}_*)$, the posterior mean and variance are

$$\hat{\mu}(\mathbf{z}_*) = \mathbf{k}_*^\top \mathbf{K}^{-1} \mathbf{y}, \quad \hat{\sigma}^2(\mathbf{z}_*) = k(\mathbf{z}_*, \mathbf{z}_*) - \mathbf{k}_*^\top \mathbf{K}^{-1} \mathbf{k}_*. \quad (10)$$

The hyperparameters $\boldsymbol{\theta} = (\sigma_f, \ell_1, \ell_2, \sigma_n)$ maximize the log marginal likelihood

$$\log p(\mathbf{y} \mid \mathbf{Z}, \boldsymbol{\theta}) = -\frac{1}{2} \mathbf{y}^\top \mathbf{K}^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}| - \frac{n}{2} \log 2\pi. \quad (11)$$

Equation (11) is maximized with normalized outputs, eight random restarts (nine optimizer runs in total), and the bounds $\sigma_f^2 \in [10^{-3}, 10^4]$, $\ell_j \in [10^{-2}, 10^3]$ and $\sigma_n^2 \in [10^{-12}, 10^{-2}]$; the initial values and all other settings are listed in the configuration block of the script. The GP was added as a third model because it provides the uncertainty estimate $\hat{\sigma}$ needed for step 9 of the assignment, and because it is the standard surrogate in aerospace design optimization [2, 12]. Whenever a hyperparameter ends at a bound, the fit is repeated with $\sigma_f^2 \leq 10^6$ and $\leq 10^8$ and the alternatives are compared by training CV only. Optimizer runs that end with an abnormal line-search termination are counted and reported.

E. Complexity Selection

The polynomial degree p of Eq. (6) is swept over $p = 1, \dots, 8$ and scored by the pooled five-fold CV RMSE on the 45 training points,

$$\text{RMSE}_{\text{CV}} = \sqrt{\frac{1}{n} \sum_{k=1}^5 \sum_{i \in \mathcal{F}_k} (y^{(i)} - \hat{f}^{(-k)}(\mathbf{x}^{(i)}))^2}, \quad (12)$$

where $\hat{f}^{(-k)}$ is refitted, scalar included, without fold $\mathcal{F}_k$ [13]. The choice of p is a bias–variance trade-off [13]: too low a degree leaves a systematic (bias) error that more data cannot remove, whereas too high a degree makes the fit sensitive to the particular sample (variance) and, numerically, to the conditioning of Ψ . Each CV training fold holds 36 points, so degree 7 ($n_{\text{terms}} = 36$) is exactly determined and degree 8 ($n_{\text{terms}} = 45$) is under-determined inside the folds. With noiseless polynomial data every degree at or above the true one gives an error at round-off level, and the arg-min among those values is decided by floating-point noise. The selection rule is therefore a parsimony rule: choose the smallest p whose CV RMSE is within 5% of the best value, or below $10^{-9} \text{std}(y_{\text{train}})$, which is treated as exact. Only the CV error of Eq. (12) enters the choice; the test and dense-grid errors are computed after the degree has been locked and are plotted only as diagnostics.

F. Validation Metrics

For n_v validation points with truth y_i and prediction $\hat{y}_i$,

$$\text{RMSE} = \sqrt{\frac{1}{n_v} \sum_i (y_i - \hat{y}_i)^2}, \quad \text{MAE} = \frac{1}{n_v} \sum_i |y_i - \hat{y}_i|, \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad (13)$$

$$\text{MaxAE} = \max_i |y_i - \hat{y}_i|, \quad \text{NRMSE} = \frac{\text{RMSE}}{\max(y_{\text{test}}) - \min(y_{\text{test}})}. \quad (14)$$

The metrics of Eqs. (13) and (14) are computed on the test set; on the dense grid, NRMSE is normalized by the range of the grid values instead. Every model is compared with the trivial baseline $\hat{y} = \text{mean}(y_{\text{train}})$; a surrogate that does not beat it has learned nothing. RMSE and MAE carry the units of f , NRMSE makes the two functions comparable, and MaxAE is the worst-case number that usually matters in design. CV is used to select, the test set gives the final

independent check, and the dense grid shows what both estimates miss.

G. Uncertainty, Sample Size, Optimization and Extrapolation

Uncertainty. The GP standard deviation $\hat{\sigma}$ of Eq. (10) is mapped over the grid and compared with the true $|f - \hat{f}|$ through the Pearson and Spearman correlations and the fraction of grid points with $|f - \hat{f}| \leq 2\hat{\sigma}$. The small diagonal jitter $\alpha = 10^{-10}$ that scikit-learn adds to $\mathbf{K}$ for numerical stability, together with σ_n^2 , puts a floor of about $\sqrt{\alpha + \sigma_n^2} \text{std}(y_{\text{train}})$ under $\hat{\sigma}$; this floor is reported with the results.

Learning curve. Each model is refitted on fresh LHS designs of $N = 20, 30, 45, 65, 90$ training points, five independent draws per size, and scored by RMSE on the full grid and on a central box whose sides are 75% of the domain sides (56% of its area). The core score separates edge effects from interior accuracy. The polynomial keeps its CV-selected degree at every N because $n_{\text{terms}} \leq 20$ in both cases; no degree reduction was needed.

Optimization. Each surrogate is minimized by bounded Nelder–Mead [14] from 20 LHS start points. Nelder–Mead is derivative free, which avoids the finite-difference failure on GP predictions noted in the reference notebook [6]. End points closer than 2% of the box width (0.08 for Rosenbrock, 0.16 for Trid) are merged into one candidate. Each distinct candidate is then evaluated once with the true function, and the predicted value, the true value and the distance to the known minimum are tabulated. The known minimum is never inserted as a result. The same multi-start search on the true function confirms the reference minimum numerically.

Extrapolation. Each model is retrained on 45 LHS points inside a reduced box (the central quarter of the domain by area) and evaluated over the full grid; errors inside and outside the training box are reported separately.

H. Reproducibility

All results come from the single script `202510630_Bahaa_Karawia.py` (Appendix A), run with Python 3.11.9, NumPy 2.4.6 [15], SciPy 1.17.1 [16], scikit-learn 1.9.1 [9], Matplotlib 3.11.2 [17] and pandas 3.0.6 [18]. The master seed is 7, and every random draw derives from it through fixed offsets set in the configuration block. The command `python 202510630_Bahaa_Karawia.py` writes every figure and table of this report to `./outputs` in about two minutes. The same unit-square design is used for both functions, so differences between Parts A and B come from the functions, not from the sampling.

The script, every generated figure and table, both versions of this report and the list of pinned package versions are available in the public repository

<https://github.com/BahaaIbrahimK/AE541-Assignment1-Surrogate-Modeling>

One command rebuilds everything from scratch: `reproduce.ps1` on Windows or `reproduce.sh` on Linux and macOS. On every change to the repository, an automated GitHub Actions job reruns the study on a clean Linux machine

and checks the fresh results against the committed ones. The workflow and parts of the code (LHS, RBF system, GP set-up) are adapted from the course notebook [6].

III. Results, Part A: Rosenbrock Function

A. Function, Domain and Minimum

Figure 1 shows the Rosenbrock function on $[-2, 2]^2$. The minimum $f = 0$ at $(1, 1)$ lies at the end of the curved “banana” valley $x_2 = x_1^2$; the script checks $f(1, 1) = 0$ before any fitting. Across the valley the function rises steeply, reaching 3609 at the corner $(-2, -2)$, while along its floor it reduces to $(1 - x_1)^2$ and varies only between 0 and 5.8. The interesting region (small f) is a thin curved band that covers a tiny fraction of the domain, so any global error metric will be dominated by the large corner values.

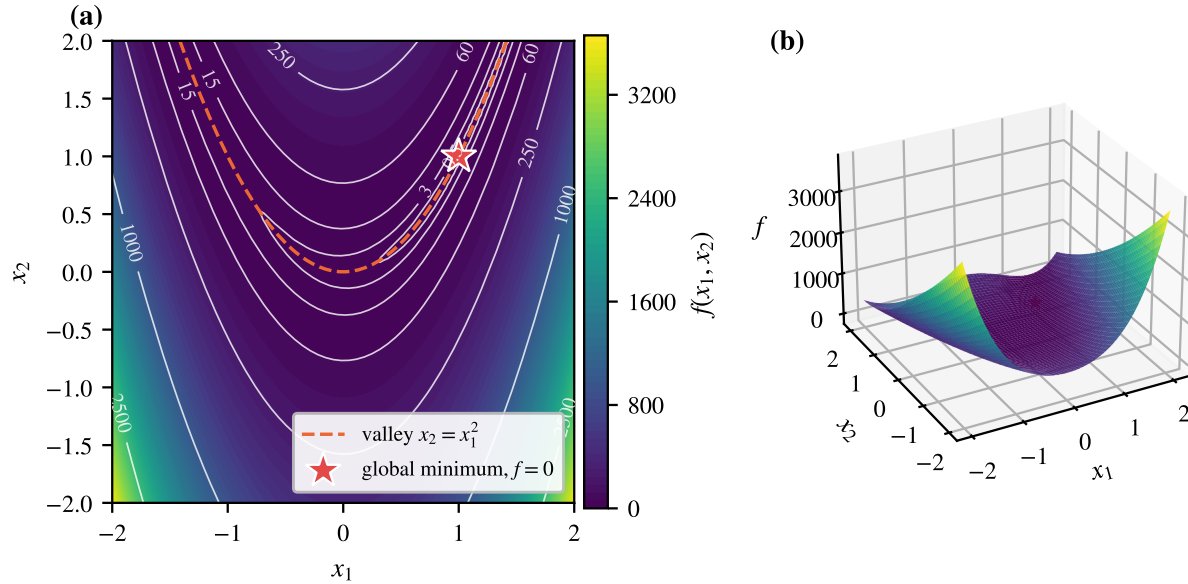


Fig. 1 Rosenbrock function on $[-2, 2]^2$. (a) Filled contour map with iso-lines at $f = 0.5, 3, 15, 60, 250, 1000, 2500$; the dashed line is the valley floor $x_2 = x_1^2$ and the star marks the global minimum $f = 0$ at $(1, 1)$. (b) Three-dimensional surface of the same field, with the minimum marked.

B. Design of Experiments

The three designs of Sec. II.B are shown in Fig. 2, and their maximin distances are listed in Table 2. The single random design leaves visible holes and clusters and its axis projections (red ticks) pile up; the LHS projections are evenly spaced by construction. Over 500 realizations, the median $\phi_{\min}$ of LHS (0.0217) is almost twice that of random sampling (0.0115), and the 5th percentile is three times larger (0.0105 against 0.0035), so the LHS rarely produces the near-duplicate points that waste a random design. The maximin LHS used here reaches $\phi_{\min} = 0.0452$, higher than every one of the 500 plain LHS draws (maximum 0.0432) and almost twice the random design at the same cost.

The budget $N = 60$ was chosen from the models to be fitted. With 45 training points a degree-4 polynomial (15 coefficients) has three points per coefficient on the full training set and 2.4 per coefficient inside a CV fold, above the usual rule of thumb $n \geq 2n_{\text{terms}}$; the learning curve of Sec. III.I then confirms that the GP error falls by two orders of magnitude up to $N = 45$ and only by a factor of three between 45 and 90, so a larger budget would buy comparatively little.

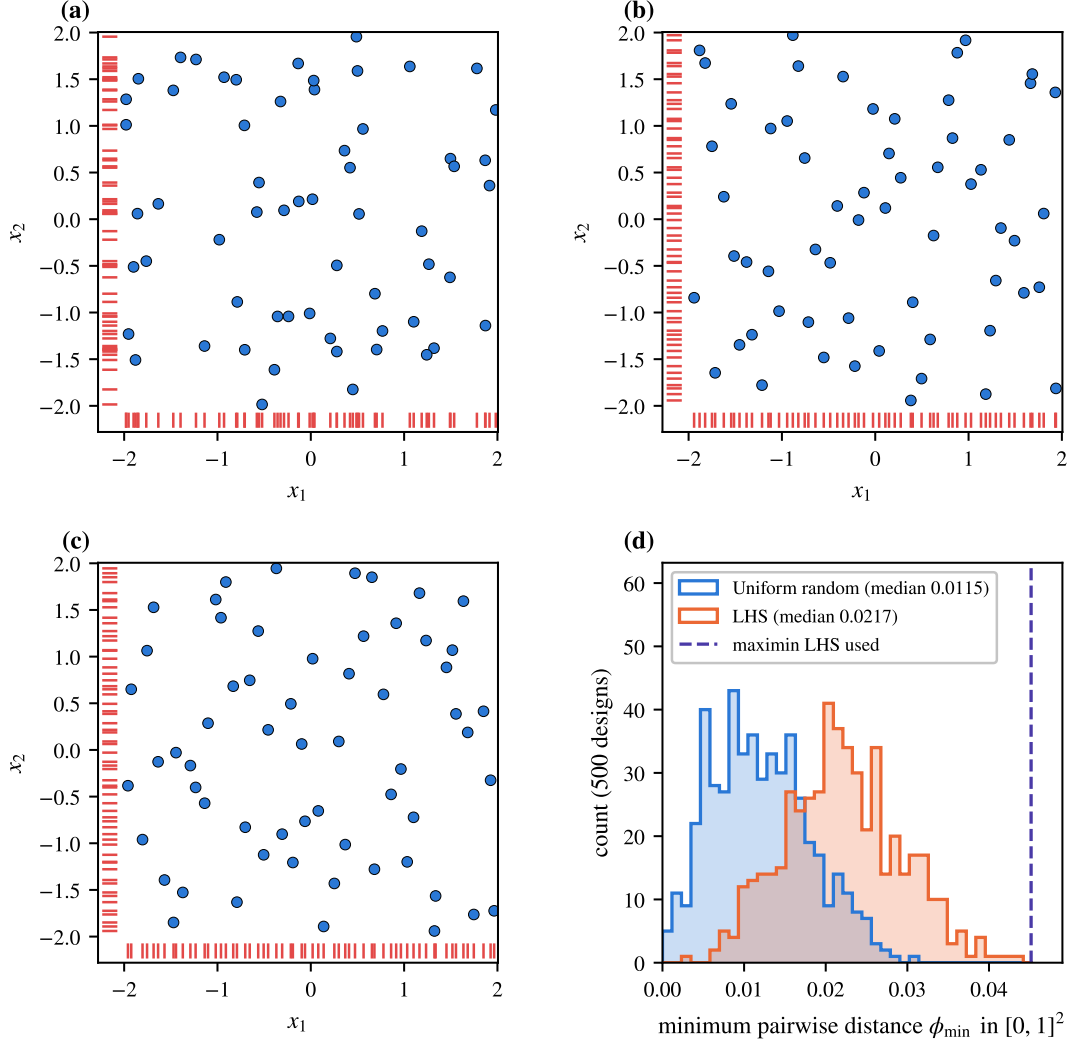


Fig. 2 Comparison of designs with the same budget $N = 60$, shown in the Rosenbrock box. (a) Uniform random, $\phi_{\min} = 0.0229$. (b) Latin hypercube, $\phi_{\min} = 0.0251$. (c) Maximin LHS (best of 100 LHS draws), $\phi_{\min} = 0.0452$, used for all results. Red ticks along the left and bottom edges are the one-dimensional projections of the points. (d) Distribution of $\phi_{\min}$, measured in the unit square, over 500 independent random and LHS designs; the dashed line is the maximin LHS actually used. The Trid study uses the same unit-square design scaled to $[-4, 4]^2$.

Table 2 Space-filling quality of the candidate designs, measured by the maximin distance $\phi_{\min}$ in the unit square ($N = 60$, higher is better).

Design	$\phi_{\min}$ (seed 7)	Median over 500 draws	5th percentile over 500 draws
Uniform random	0.0229	0.0115	0.0035
Latin hypercube	0.0251	0.0217	0.0105
Maximin LHS (used)	0.0452	—	—

C. Train/Test Split

Figure 3 shows the 45 training and 15 test points. The test points are spread over the whole box. Three lie on the valley floor, at $(-0.83, 0.68)$, $(-0.46, 0.22)$ and $(-0.10, 0.06)$, where $f < 3.4$. Two, $(0.91, 1.36)$ and $(1.45, 0.88)$, lie within 0.5 of the minimum but on the steep flanks ($f = 27$ and 151), not in the low- f valley near it, and one, $(-1.47, -1.85)$, approaches the corner $(-2, -2)$. Fifteen points cannot sample every feature, which is one reason CV and the dense-grid diagnostic are reported alongside the test metrics.

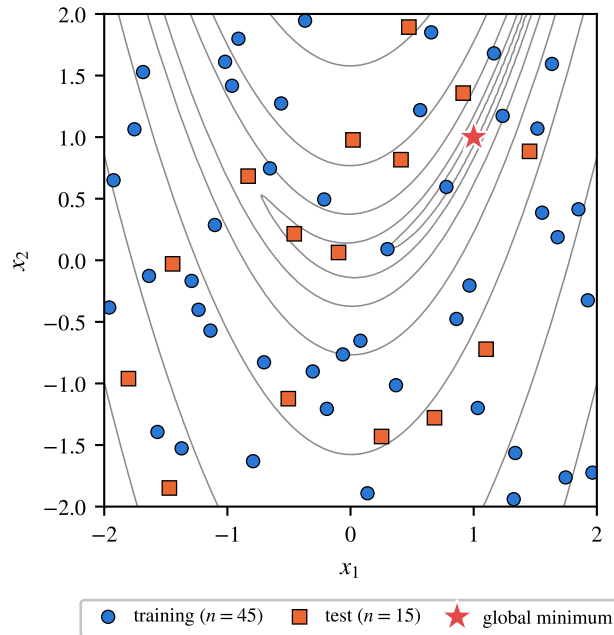


Fig. 3 Rosenbrock: locations of the 45 training points and the 15 held-out test points over iso-lines of f ; the star is the global minimum.

D. Polynomial Degree Selection

The degree sweep is shown in Fig. 4 and listed in Table 3. For $p \leq 3$ the train, CV and test errors are all large (111 to 607) and close to one another: the model is too rigid and the error is bias, which more data would not remove. At $p = 4$ every curve collapses by about 14 orders of magnitude to round-off ($\text{RMSE}_{\text{CV}} = 1.27 \times 10^{-12}$), because the hypothesis space now contains the truth. The plain CV arg-min would have picked $p = 5$, whose CV RMSE is lower by

only 1.4×10^{-13} , a floating-point difference; the parsimony rule selects $p = 4$, the true degree, from the data alone. The recovered physical coefficients match Eq. (2) to within 3.8×10^{-13} , and every coefficient that should vanish is below 2.5×10^{-13} .

Beyond $p = 4$ no classic over-fitting appears in the *train* or *test* error: the data are noiseless and the truth is inside every larger space, so the extra coefficients are simply fitted as zero. What does appear is numerical. The condition number of the standardized design matrix grows from 33 at $p = 4$ to 2.3×10^3 at $p = 7$ and 2.1×10^5 at $p = 8$, and the CV error climbs to 2.6×10^{-11} at $p = 7$ (36 terms on 36 fold points) and jumps to 82.9 at $p = 8$. At $p = 8$ each fold has 45 unknowns and 36 equations: the minimum-norm solution passes exactly through the fold's points but is not the true polynomial, and it is badly wrong on the left-out fold. On the full training set the same degree has a square 45×45 system whose unique solution is the truth again (test RMSE 1.9×10^{-10}). The gap between the CV curve and the train curve is therefore a signal of under-determination, not of noise-driven over-fitting.

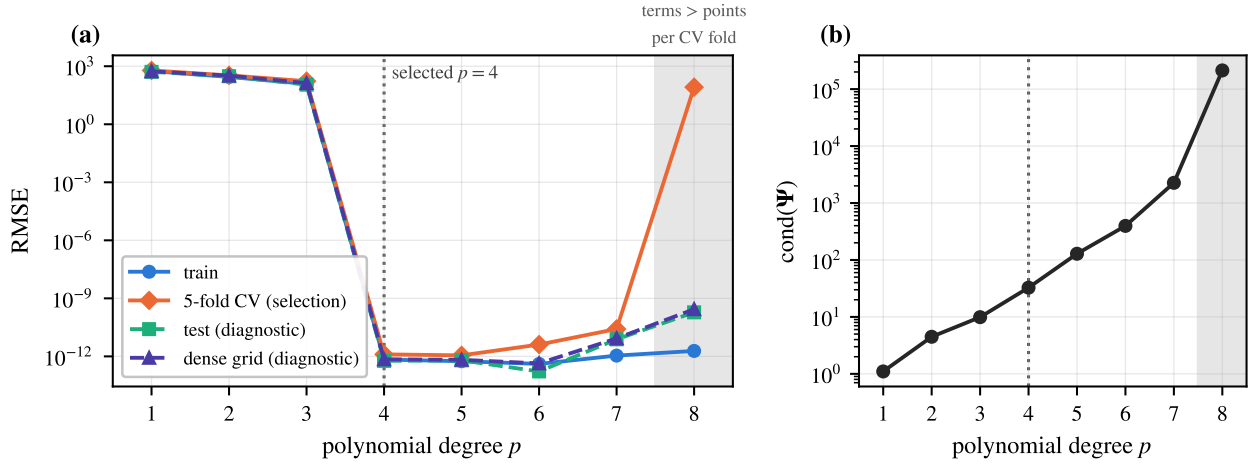


Fig. 4 Rosenbrock: polynomial complexity study. (a) RMSE against degree p : training error, five-fold CV error on the training set (the only curve used for selection), and the held-out test and dense-grid errors added afterwards as diagnostics; the dotted line marks the selected $p = 4$ and the shaded band marks degrees with more coefficients than points in a CV training fold. (b) Condition number of the standardized monomial design matrix.

E. Held-Out Validation and Cross-Validation

Table 4 lists the test metrics and Table 5 the CV and dense-grid errors; the parity plots are in Fig. 5. All three models beat the mean baseline, whose R^2 is even negative (-0.069) because the test mean differs from the training mean. The polynomial is exact to round-off. The GP reaches a test RMSE of 0.0138 (NRMSE 7.8×10^{-6}) with a worst test error of 0.047. The cubic RBF has a test RMSE of 33.3 and a MaxAE of 116, but its $R^2 = 0.9962$ still looks excellent: the denominator of R^2 is the huge variance of the corner values, which is exactly why R^2 alone flatters a model on a function with this dynamic range.

The three error estimates in Table 5 do not agree, and the disagreement is informative. For the RBF the CV RMSE

Table 3 Polynomial degree sweep on the 45 training points: number of coefficients, condition number of the standardized design matrix, and five-fold CV RMSE for both functions. Selected degrees in bold.

p	n_{terms}	Points per fold	cond(Ψ)	CV RMSE, Rosenbrock	CV RMSE, Trid
1	3	36	1.10	6.07×10^2	1.01×10^1
2	6	36	4.48	3.36×10^2	1.04×10^{-14}
3	10	36	9.90	1.70×10^2	2.89×10^{-14}
4	15	36	32.6	1.27×10^{-12}	1.82×10^{-14}
5	21	36	129	1.13×10^{-12}	2.58×10^{-14}
6	28	36	396	4.09×10^{-12}	3.37×10^{-14}
7	36	36	2.26×10^3	2.61×10^{-11}	5.56×10^{-12}
8	45	36	2.13×10^5	8.29×10^1	9.17×10^{-1}

(78.2) is more than twice the test RMSE (33.3), and the dense grid (66.1) sits much closer to CV. The 15 test points happen to miss the worst errors, which lie in the corner itself and on the flanks, while CV predicts all 45 training points from models built on 36, which is more pessimistic but closer to the truth here. A single small test set can make a surrogate look better than it is, which is why CV and the error maps are needed alongside it.

Table 4 Held-out test metrics (15 points never used in fitting or selection). The range of y_{test} used for NRMSE is 1780.9 for Rosenbrock and 28.07 for Trid.

Function	Model	RMSE	MAE	R^2	MaxAE	NRMSE
Rosenbrock	Baseline, mean(y_{train})	558.4	458.8	-0.069	1270.2	0.314
	Polynomial, $p = 4$	5.87×10^{-13}	5.22×10^{-13}	1.000000	8.98×10^{-13}	3.30×10^{-16}
	Cubic RBF	33.27	18.21	0.996204	115.55	1.87×10^{-2}
	GP	1.38×10^{-2}	7.48×10^{-3}	1.000000	4.70×10^{-2}	7.75×10^{-6}
Trid	Baseline, mean(y_{train})	9.725	8.375	-0.262	15.12	0.347
	Polynomial, $p = 2$	7.91×10^{-15}	6.90×10^{-15}	1.000000	1.55×10^{-14}	2.82×10^{-16}
	Cubic RBF	4.25×10^{-2}	2.04×10^{-2}	0.999976	0.155	1.51×10^{-3}
	GP	2.91×10^{-5}	1.87×10^{-5}	1.000000	8.98×10^{-5}	1.04×10^{-6}

Table 5 Five-fold CV RMSE on the training partition (used for selection), held-out test RMSE (final check), and dense-grid diagnostics over 14 400 points (available only because the truth is known). Grid NRMSE is normalized by the range of f on the grid.

Function	Model	CV RMSE	Test RMSE	Grid RMSE	Grid MaxAE	Grid NRMSE
Rosenbrock	Polynomial, $p = 4$	1.27×10^{-12}	5.87×10^{-13}	7.06×10^{-13}	3.13×10^{-12}	1.96×10^{-16}
	Cubic RBF	78.24	33.27	66.12	917.8	1.83×10^{-2}
	GP	5.78×10^{-2}	1.38×10^{-2}	3.27×10^{-2}	0.699	9.06×10^{-6}
Trid	Polynomial, $p = 2$	1.04×10^{-14}	7.91×10^{-15}	8.14×10^{-15}	2.84×10^{-14}	1.57×10^{-16}
	Cubic RBF	0.178	4.25×10^{-2}	0.156	2.72	2.99×10^{-3}
	GP	4.69×10^{-5}	2.91×10^{-5}	5.58×10^{-5}	9.57×10^{-4}	1.07×10^{-6}

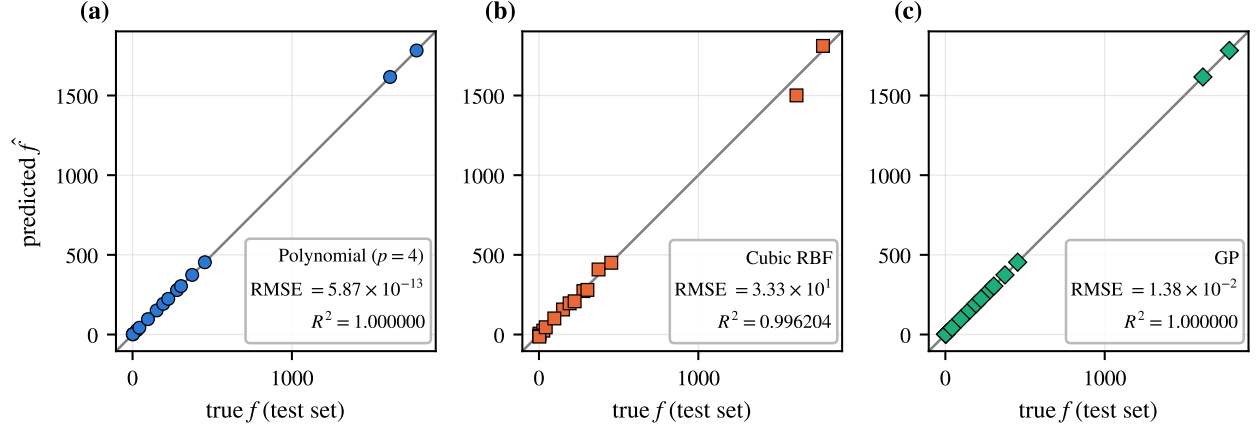


Fig. 5 Rosenbrock: parity plots of predicted against true f on the 15 held-out test points for (a) the degree-4 polynomial, (b) the cubic RBF and (c) the GP. The solid line is perfect prediction; the inset gives the test RMSE and R^2 .

F. GP Hyperparameters and the Banana Valley

The fitted length scales are $\ell_1 = 2.57$ and $\ell_2 = 23.4$ in standardized units, or 3.11 and 27.4 in physical units. The ratio $\ell_2/\ell_1 \approx 8.8$ says that the function is roughly an order of magnitude more sensitive to x_1 than to x_2 , which agrees with Eq. (2): x_1 enters through the quartic $100x_1^4$ and the coupling $-200x_1^2x_2$, whereas x_2 enters only quadratically. The ARD kernel, however, measures sensitivity only along the coordinate axes. The valley is curved: inside the box it exists for $|x_1| \leq \sqrt{2}$, and its tangent direction $(1, 2x_1)$ rotates from $(1, -2.83)$ to $(1, 2.83)$ along it, so no pair of axis-aligned length scales can describe “slow along the valley, fast across it”. The two fitted values are a compromise averaged over the whole box, not a description of the valley.

The signal variance ended at its upper bound $\sigma_f^2 = 10^4$ and the noise variance close to its lower bound (1.2×10^{-12} against 10^{-12}), and 3 of the 9 optimizer runs ended with an abnormal line-search termination; the run with the highest likelihood is kept. A stationary squared-exponential kernel approaches a polynomial only in the limit of large amplitude and long length scales [11], so the likelihood keeps pushing σ_f^2 and ℓ_j upward, into a flat and poorly conditioned region. Table 6 shows that widening the bound does not settle the matter. With $\sigma_f^2 \leq 10^6$ the likelihood rises to 214.7, but the training CV error gets worse (0.079 against 0.058). With $\sigma_f^2 \leq 10^8$ the likelihood falls to 183.6, which cannot be the global optimum, because the 10^4 solution is still admissible: the optimizer has found a different local optimum. The marginal likelihood is therefore multimodal and unreliable here, and the choice between the fits was made by training CV, which favors the 10^4 bound. The bound acts as a mild regularizer. The principled remedy would be universal Kriging, a GP around a polynomial trend [2], which is beyond the scope of this assignment.

Table 6 GP hyperparameters refitted with wider bounds on the normalized signal variance σ_f^2 (physical length scales). The alternatives are judged by training CV only; the first row of each block is the model used, for which 3 (Rosenbrock) and 8 (Trid) of the 9 optimizer runs ended with an abnormal line-search termination. A wider bound still admits the narrower optimum, so a lower likelihood indicates a different local optimum, not a worse model class.

Function	Bound on σ_f^2	Fitted σ_f^2	ℓ_1	ℓ_2	σ_n^2	$\log p(\mathbf{y} \boldsymbol{\theta})$	CV RMSE
Rosenbrock	10^4 (used)	10^4 (at bound)	3.11	27.4	1.2×10^{-12}	197.8	5.78×10^{-2}
	10^6	10^6 (at bound)	4.80	99.8	9.8×10^{-10}	214.7	7.89×10^{-2}
	10^8	3.77×10^6	6.92	126.4	4.8×10^{-8}	183.6	6.10×10^{-2}
Trid	10^4 (used)	10^4 (at bound)	37.5	37.2	1.9×10^{-12}	302.6	4.69×10^{-5}
	10^6	10^6 (at bound)	125.6	140.9	9.6×10^{-9}	260.1	2.74×10^{-4}
	10^8	1.14×10^5	48.8	51.8	6.7×10^{-9}	254.6	9.34×10^{-5}

G. Side-by-Side Visual Comparison

Figures 6 to 8 give the mandatory side-by-side comparison for the CV-best surrogate (the polynomial) and for the other two models; Fig. 9 gives the 3-D pair for the best surrogate. The polynomial error field (right panel of Fig. 6) is pure round-off, at most 3.1×10^{-12} against a signal of 3609. Its smooth large-scale pattern reflects floating-point cancellation in the quartic terms, not model error. Because this map carries no modeling information, the GP and RBF maps are where the failure patterns can be read. On every map of f and $\hat{f}$ in this report, the color scale extends slightly below the minimum of f on the grid (by 0.1% of the range of f , i.e. to -3.6 for Rosenbrock and -2.05 for Trid), and surrogate values below that limit are drawn in red, so a clearly impossible prediction cannot hide in the darkest color.

For the GP (right panel of Fig. 7) the error is below 0.05 almost everywhere and concentrates in the corner $(-2, -2)$, where it reaches 0.70. That corner combines the steepest part of the surface ($f = 3609$) with the largest corner gap in the design: its nearest training point is 0.74 away, against 0.28 to 0.57 for the other three corners, and no point lies beyond the boundary to constrain the fit. For the RBF (right panel of Fig. 8) the error is three orders of magnitude larger and has the same hot spot, reaching 918 in the corner $(-2, -2)$. It also shows negative lobes of several hundred on both steep flanks near $(\pm 1.7, -0.9)$, where the interpolant overshoots between samples, and the surrogate panel shows closed contours inside the valley that do not exist in the truth, enclosing a red region where $\hat{f} < -3.6$ and falls to -16.8 , although $f \geq 0$ everywhere. A cubic spline with a linear tail cannot reproduce a quartic, and near the boundary it relaxes toward its linear tail, whereas the true surface curves up fastest precisely at the edges [19]. In all maps the largest errors sit in gaps between samples or outside the convex hull of the training points, never at the samples themselves.

H. Uncertainty Quantification

Figure 10 compares the GP standard deviation $\hat{\sigma}$, which is available without knowing the truth, with the actual error $|f - \hat{f}|$, which is not. The two maps share their structure: both are smallest near the samples and largest in the corners,

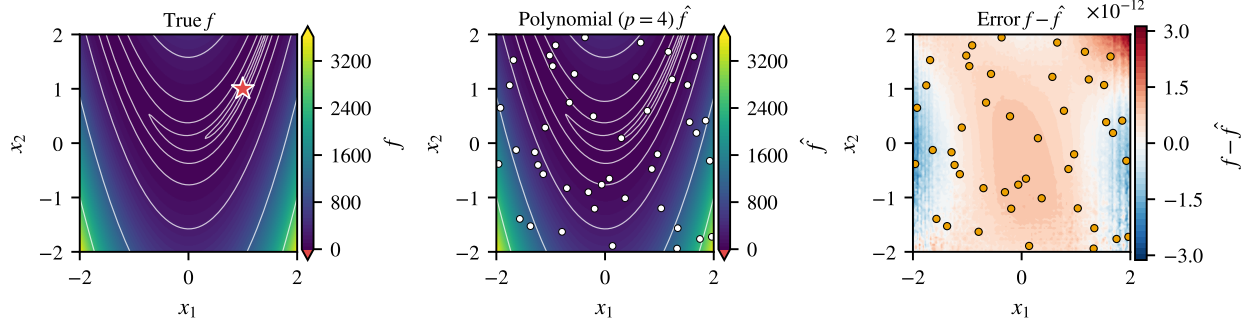


Fig. 6 Rosenbrock, best surrogate by CV (degree-4 polynomial). Left: true f on the 120×120 grid with the minimum marked. Center: surrogate $\hat{f}$ on the same grid with identical color limits, training points overlaid. Right: error $f - \hat{f}$ on a diverging scale centered at zero with training points overlaid; the maximum $|e|$ is 3.1×10^{-12} (round-off). White iso-lines are drawn at the same levels in the first two panels.

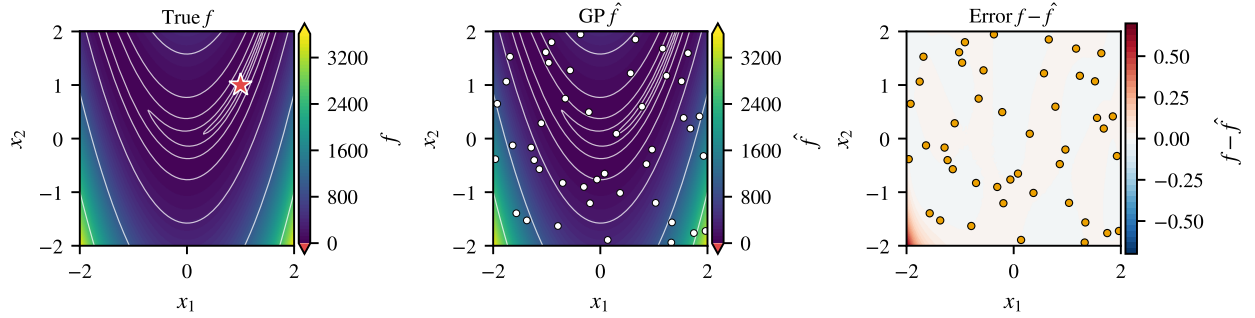


Fig. 7 Rosenbrock, GP surrogate: true f , surrogate $\hat{f}$ with identical color limits, and error $f - \hat{f}$ (diverging scale centered at zero, maximum $|e| = 0.699$ in the corner $(-2, -2)$), with training points overlaid on the surrogate and error panels.

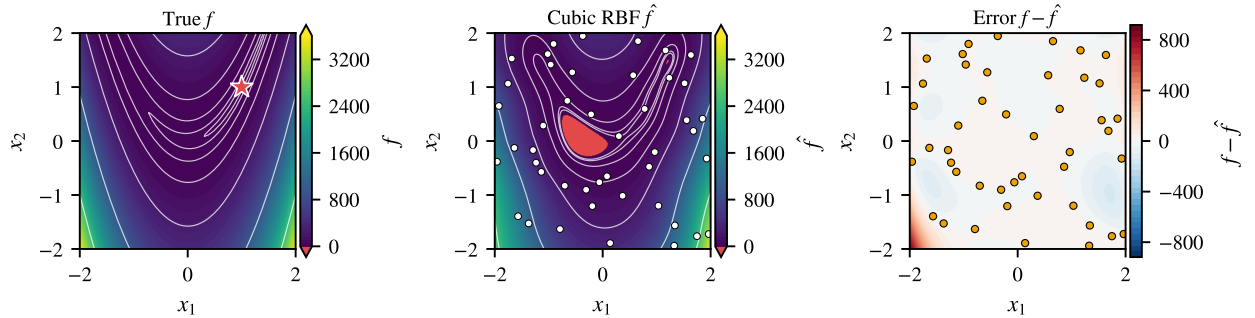


Fig. 8 Rosenbrock, cubic RBF surrogate: true f , surrogate $\hat{f}$ with identical color limits, and error $f - \hat{f}$ (diverging scale centered at zero, maximum $|e| = 918$ in the corner $(-2, -2)$), with training points overlaid. Negative lobes on both steep flanks mark overshoot between samples. The red regions in the center panel are where the surrogate predicts values more than 0.1% of the range of f below its minimum ($\hat{f} < -3.6$): a large one reaching -16.8 inside closed contours that the truth does not have, and a small one near $(1.2, 1.45)$ in the upper arm of the valley.

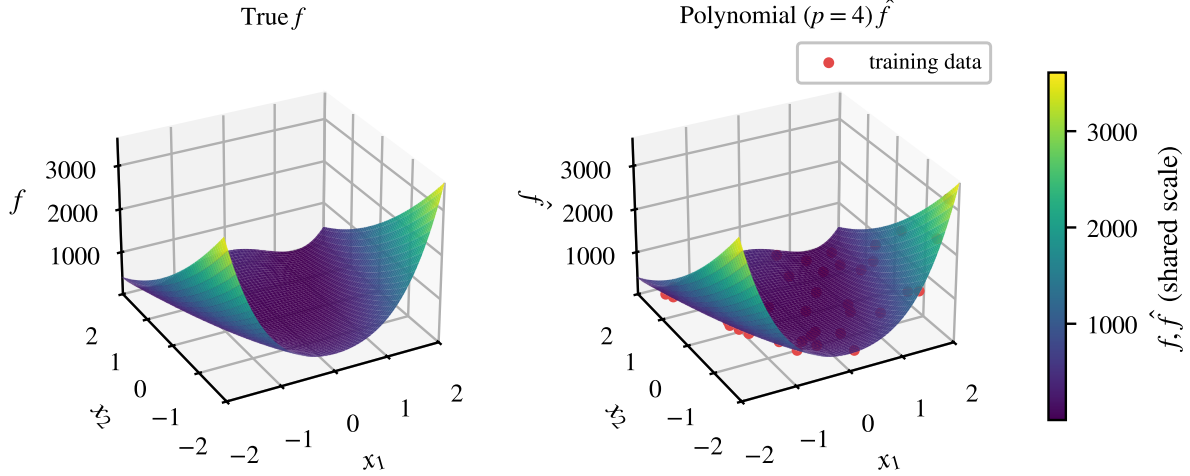


Fig. 9 Rosenbrock: 3-D surfaces of the true function and of the degree-4 polynomial surrogate with identical axis limits, color limits and viewing angle; red dots are the training data.

especially $(-2, -2)$. Over the grid, the Pearson correlation is 0.70 and the Spearman rank correlation 0.67, and 97.8% of grid points satisfy $|f - \hat{f}| \leq 2\hat{\sigma}$. The 2.2% outside the band matter most: they lie on the left side of the domain (centroid $(-1.50, -0.61)$) and include the worst error of all, 0.70, which exceeds twice the largest predicted standard deviation ($2\hat{\sigma}_{\max} = 0.50$). The band therefore fails exactly where the error is largest. Away from the corners, $\hat{\sigma}$ is close to its numerical floor of 6.8×10^{-3} (median 5.9×10^{-3} in the central box, the same order as the value 7.0×10^{-3} at the training points), so the useful information in $\hat{\sigma}$ lies in its rise toward the edges, up to 0.25. The two quantities are related because both grow with distance from the data. They are not the same quantity: $\hat{\sigma}$ depends only on the sample locations and the fitted kernel, not on the observed values at the prediction point, so it measures how poorly constrained the model is under its own assumptions, not how wrong it actually is. Panel (c) shows this directly: at a given $\hat{\sigma}$ the true error spreads over two to three decades, and a lower branch of points has errors ten times smaller than $\hat{\sigma}$. The GP is a useful *guide* to where to sample next, but not a certificate of accuracy.

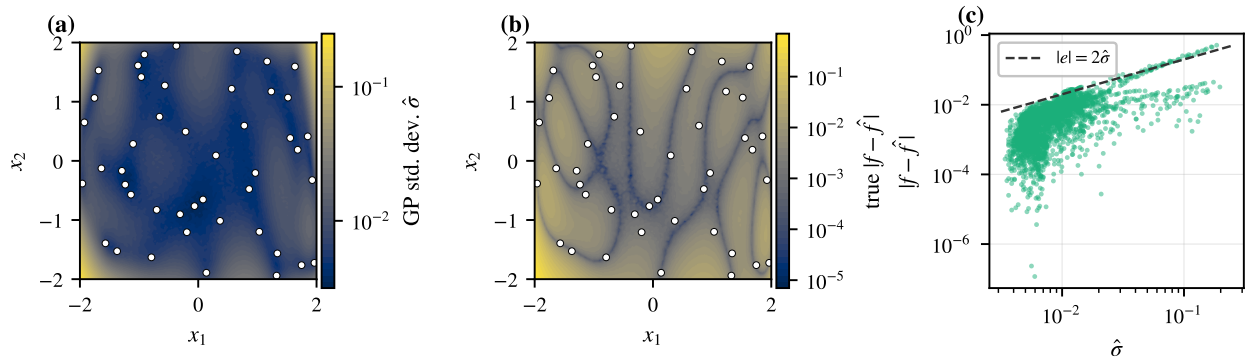


Fig. 10 Rosenbrock, GP uncertainty. (a) Predicted standard deviation $\hat{\sigma}$ (log color scale), computable without the truth. (b) True absolute error $|f - \hat{f}|$ (log color scale). (c) $|f - \hat{f}|$ against $\hat{\sigma}$ for 3000 random grid points on logarithmic axes; the dashed line is $|e| = 2\hat{\sigma}$. White dots in (a) and (b) are the training points.

I. Effect of Sample Size

The learning curves are shown in Fig. 11. The polynomial stays at round-off ($\sim 10^{-12}$) at every N , because 20 points already over-determine its 15 coefficients. The GP improves fastest, from a median grid RMSE of 2.91 at $N = 20$ to 0.029 at $N = 45$ and 0.0098 at $N = 90$: going from 20 to 45 points buys a factor of 100, going from 45 to 90 only a factor of three. The RBF improves slowly, from 162 to 27.5 over the same range; in the central box it falls faster, from 61 to 3.8. Sampling variability is large at small N : at $N = 30$ the five GP runs span a factor of 19 (0.093 to 1.79), which is a strong reason not to report a single-seed number as “the” accuracy of a model.

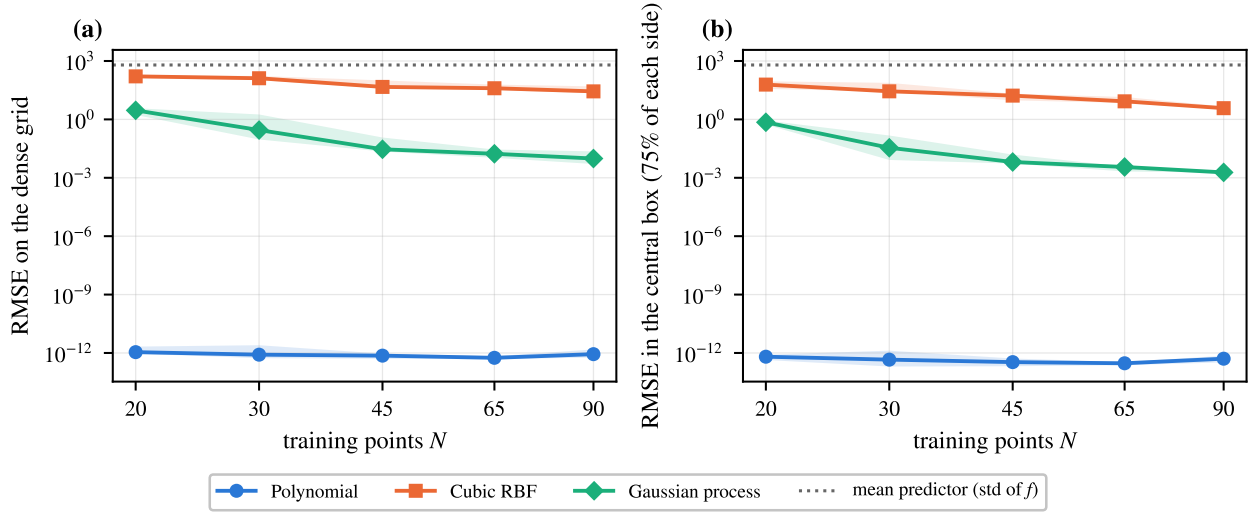


Fig. 11 Rosenbrock: learning curves on logarithmic axes. (a) RMSE over the full 120×120 grid. (b) RMSE over the central box whose sides are 75% of the domain sides. Lines are medians over five independent LHS designs per training size and shaded bands span their minimum and maximum; the dotted line is the error of predicting the mean (standard deviation of f on the grid, 624).

J. Surrogate-Based Optimization and Verification

Table 7 and Fig. 12 give the optimization results. All 20 starts on the polynomial converge to $(1.000000, 1.000000)$, and the true function there is 7.5×10^{-16} . All 20 starts on the GP converge to $(0.9940, 0.9883)$, only 0.013 from the true minimum, with a true value of 4.5×10^{-5} against a predicted 6.1×10^{-4} . The GP’s positional error, -0.0060 in x_1 and -0.0117 in x_2 , has a slope of 1.95. That is almost exactly the slope of the valley tangent, $2x_1 = 2$ at $x_1 = 1$. The surrogate found the valley, but not the precise point along its nearly flat floor, which is exactly the difficulty that makes Rosenbrock a benchmark.

The RBF shows why verification is not optional. Its best candidate, reached by 11 of the 20 starts, is a *spurious* minimum at $(-0.44, 0.04)$ with a predicted $\hat{f} = -16.8$: negative, although $f \geq 0$ everywhere, and 1.73 away from the true minimum, where the true value is 4.50. It is an undershoot of the interpolant between samples inside the valley. The candidate closest to the truth, $(0.94, 0.87)$ with $f = 0.031$, ranks only second on the surrogate. Trusting the surrogate’s

own ranking would have picked the wrong design; one true evaluation per candidate, six in total, exposes the error. A second, small undershoot patch near $(1.2, 1.45)$ ($\hat{f} < -3.6$, Fig. 12b) was not reached by any start, so the multi-start search found four of at least five RBF minima, a reminder that 20 starts do not guarantee a complete search.

Table 7 Surrogate-based optimization (20 bounded Nelder–Mead starts per surrogate) and verification with the true function. “Hits” is the number of starts that ended in each cluster; d is the distance to the known minimum. The last row of each block is the same multi-start search on the true function.

Function	Surrogate	Hits	$\hat{\mathbf{x}}^*$	Predicted $\hat{f}(\hat{\mathbf{x}}^*)$	True $f(\hat{\mathbf{x}}^*)$	$ \hat{f} - f $	d
Rosenbrock	Polynomial	20	(1.0000, 1.0000)	-5.0×10^{-13}	7.5×10^{-16}	5.1×10^{-13}	2.1×10^{-9}
	RBF, cluster 1	11	(−0.4388, 0.0366)	−16.77	4.503	21.27	1.732
	RBF, cluster 2	3	(0.9425, 0.8718)	−1.761	0.0307	1.792	0.140
	RBF, cluster 3	3	(0.7209, 0.4026)	−0.797	1.449	2.247	0.659
	RBF, cluster 4	3	(−1.1492, 1.3781)	8.312	4.948	3.364	2.182
	GP	20	(0.9940, 0.9883)	6.1×10^{-4}	4.5×10^{-5}	5.7×10^{-4}	1.3×10^{-2}
	True f (check)	20	(1.0000, 1.0000)	–	1.9×10^{-20}	–	2.6×10^{-10}
Trid	Polynomial	20	(2.0000, 2.0000)	−2.000000	−2.000000	1.3×10^{-14}	1.0×10^{-8}
	RBF	20	(1.9950, 1.9933)	−2.012319	−1.999964	1.2×10^{-2}	8.4×10^{-3}
	GP	20	(1.9995, 1.9997)	−2.000000	−2.000000	2.7×10^{-7}	5.7×10^{-4}
	True f (check)	20	(2.0000, 2.0000)	–	−2.000000	–	1.1×10^{-8}

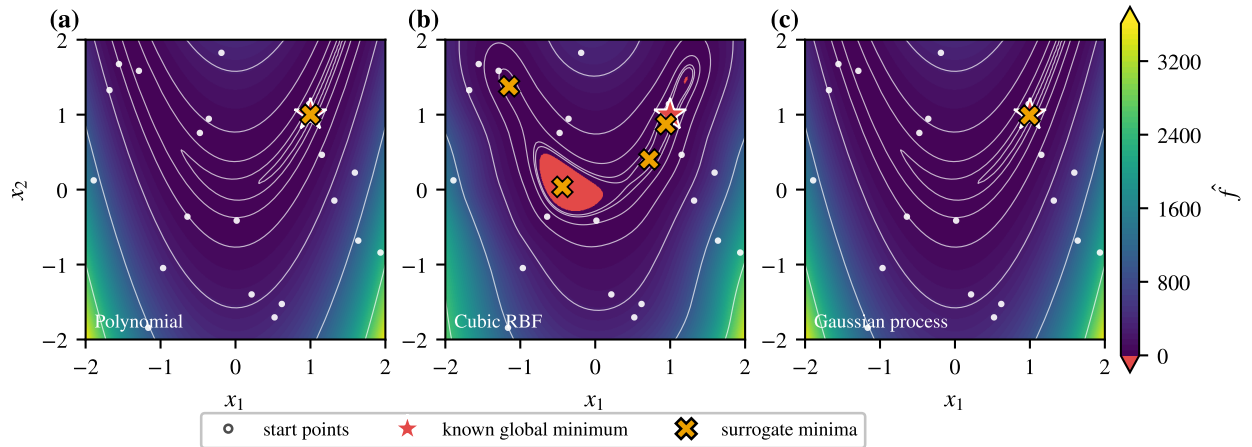


Fig. 12 Rosenbrock: multi-start optimization on (a) the degree-4 polynomial, (b) the cubic RBF and (c) the GP. The background is each surrogate $\hat{f}$ with the color limits of the true function and white iso-lines of $\hat{f}$; white dots are the 20 start points, crosses the distinct surrogate minima, and the star the known global minimum. Four distinct RBF minima were found from the 20 starts, including the spurious deepest one at $(-0.44, 0.04)$, which lies inside the large red region where $\hat{f} < -3.6$. The small red patch near $(1.2, 1.45)$ is a further undershoot that no start reached.

K. Extrapolation

Table 8 and Fig. 13 give the reduced-box experiment (training on $[-1, 1]^2$). Inside the box the GP error is small (RMSE 0.0064); outside it rises to 14.9, a factor of 2320, with a worst error of 121 in the corners, where the true

function reaches 3609 but the GP has no data to follow the quartic growth. The RBF, whose linear tail extrapolates linearly, is worse in absolute terms (574 outside). The polynomial stays exact outside the box, but only because its assumed form happens to be the true one. In a real problem that is unknowable, and a surrogate should never be trusted outside its training box. Note also that the true minimum (1, 1) lies on the corner of this training box, so any optimizer that stops at the edge of the DOE box is telling the engineer to re-sample, not to trust the answer.

Table 8 Extrapolation test: each model is trained on 45 LHS points in the central sub-box $([-1, 1]^2$ for Rosenbrock, $[-2, 2]^2$ for Trid) and evaluated over the full grid. The last column is the ratio of the RMSE outside to the RMSE inside the training box.

Function	Model	RMSE inside	RMSE outside	MaxAE inside	MaxAE outside	Ratio out/in
Rosenbrock	Polynomial	5.9×10^{-14}	8.3×10^{-13}	2.0×10^{-13}	3.6×10^{-12}	14
	Cubic RBF	6.07	574.2	66.4	2622	95
	GP	6.4×10^{-3}	14.93	0.101	120.6	2321
Trid	Polynomial	2.8×10^{-15}	1.2×10^{-14}	8.9×10^{-15}	4.3×10^{-14}	4.4
	Cubic RBF	5.3×10^{-2}	3.712	0.831	19.12	70
	GP	9.9×10^{-6}	2.8×10^{-3}	1.7×10^{-4}	2.6×10^{-2}	287

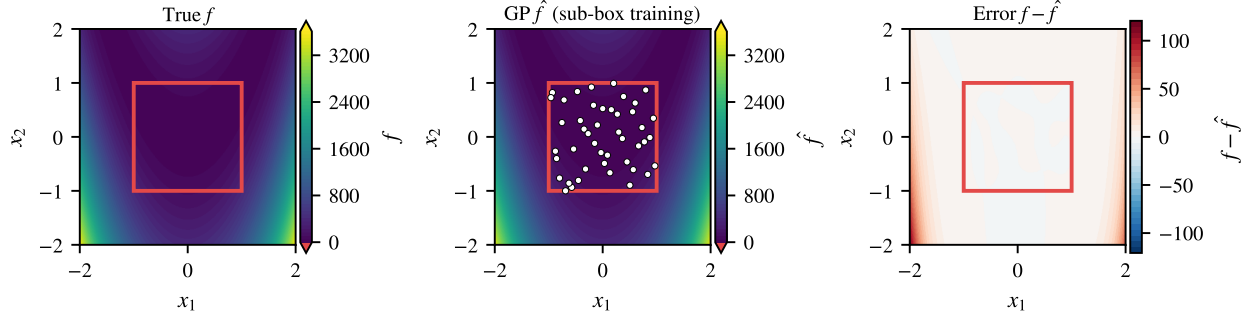


Fig. 13 Rosenbrock, extrapolation of a GP trained only inside the red box $[-1, 1]^2$ (white dots). Left: true f . Center: GP prediction over the full domain with the color limits of the true function (values above them saturate; no value falls below the red undershoot limit). Right: error $f - \hat{f}$, which grows by three orders of magnitude outside the training box.

IV. Results, Part B: Trid Function

A. Function, Domain and Analytical Minimum

The two-variable Trid function, Eq. (3), is shown in Fig. 14. Its minimum can be found by hand. Setting the gradient to zero,

$$\nabla f = \begin{bmatrix} 2(x_1 - 1) - x_2 \\ 2(x_2 - 1) - x_1 \end{bmatrix} = \mathbf{0} \implies \underbrace{\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}}_{\mathbf{H}} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}. \quad (15)$$

Adding the two equations gives $x_1 + x_2 = 4$; subtracting them gives $3(x_1 - x_2) = 0$. Hence

$$\mathbf{x}^* = (2, 2), \quad f(\mathbf{x}^*) = (2 - 1)^2 + (2 - 1)^2 - 2 \cdot 2 = -2. \quad (16)$$

The Hessian $\mathbf{H}$ is constant, with eigenvalues 1 and 3 and eigenvectors along $(1, 1)$ and $(1, -1)$. It is positive definite, so f is strictly convex and $(2, 2)$ is its unique global minimum on any domain containing it. The level sets are ellipses tilted at 45° with axis ratio $\sqrt{3}$, elongated along $(1, 1)$, the direction of the smaller eigenvalue (Fig. 14a). On $[-4, 4]^2$ the function ranges from -2 to 50 , the maximum occurring at the corners $(-4, 4)$ and $(4, -4)$, where the cross term $-x_1x_2$ is largest and positive.

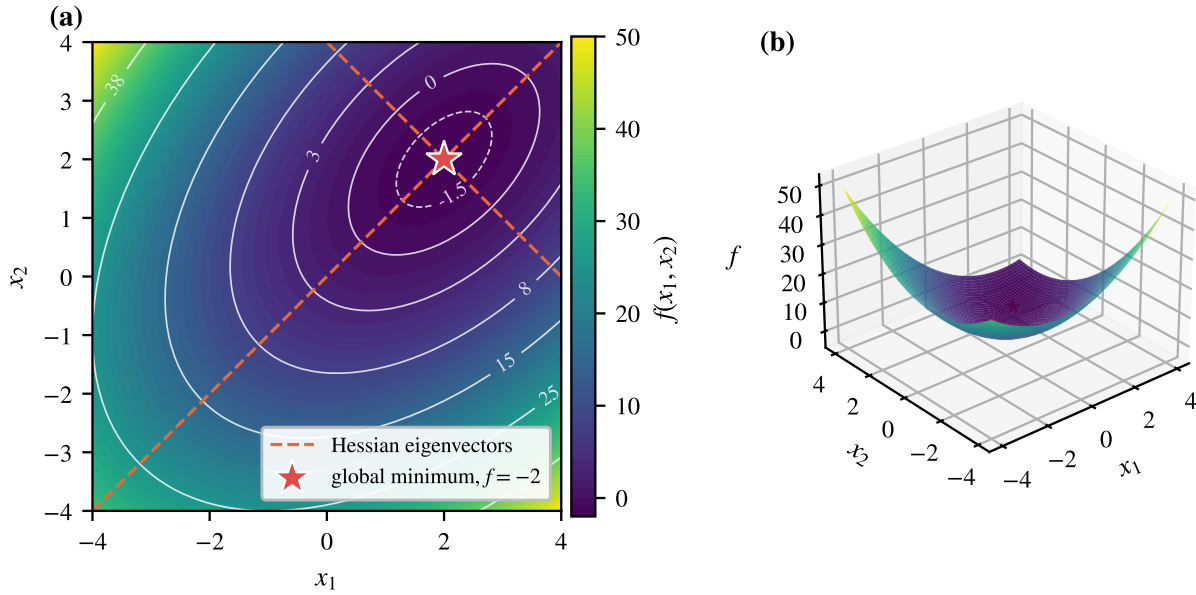


Fig. 14 Trid function on $[-4, 4]^2$. (a) Filled contour map with iso-lines at $f = -1.5, 0, 3, 8, 15, 25, 38$; dashed lines are the Hessian eigenvector directions $(1, 1)$ and $(1, -1)$ through the minimum, and the star is the global minimum $f = -2$ at $(2, 2)$. (b) Three-dimensional surface with the minimum marked.

B. Design of Experiments and Split

The Trid study uses the same maximin LHS as Part A (Fig. 2c and Table 2), scaled to $[-4, 4]^2$. Its $\phi_{\min} = 0.0452$ in unit coordinates corresponds to 0.36 in physical units, and the characteristic spacing $\sqrt{|X|/n}$ of the 45 training points is 1.19. That spacing is adequate here because the Trid function has no short-wavelength features: its curvature is constant, and a quadratic is fully determined by six well-placed values. The same argument justifies $N = 60$: ten times the six coefficients of the exact model, with 36 points per CV fold. The split is shown in Fig. 15; the test points cover all four quadrants, and two of them, $(1.83, 2.72)$ and $(2.91, 1.77)$, lie 0.74 and 0.94 from the minimum.

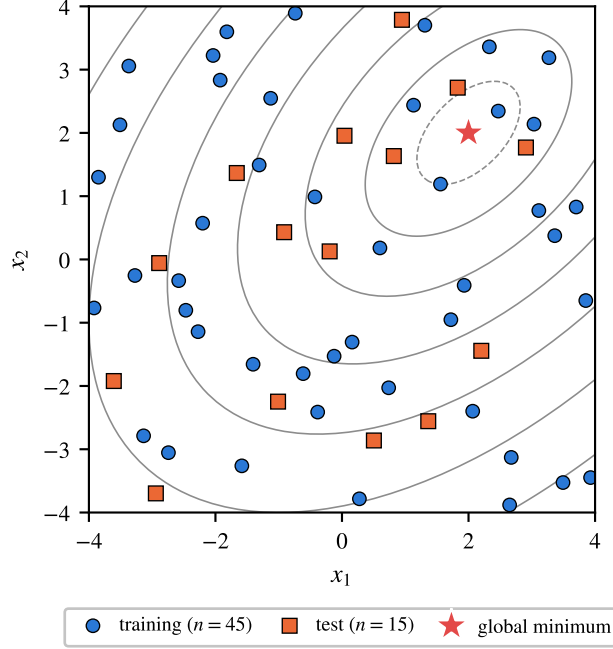


Fig. 15 Trid: locations of the 45 training points and the 15 held-out test points over iso-lines of f ; the star is the global minimum.

C. Polynomial Degree Selection

The Trid degree sweep (Fig. 16, Table 3) repeats the Rosenbrock pattern at a lower degree. Degree 1 under-fits: a plane cannot represent a bowl, and its train, CV and test RMSE (9.68, 10.1 and 6.22) are almost equal to the spread of the data itself, a pure bias error. At $p = 2$ the CV RMSE drops to 1.04×10^{-14} , 15 orders of magnitude lower, and the parsimony rule selects the true degree. Degrees 3 to 6 stay at round-off, degree 7 rises to 5.6×10^{-12} with the condition number, and degree 8 jumps to 0.92 because the folds are under-determined, as explained for Part A. The recovered physical coefficients match Eq. (3) to within 1.2×10^{-14} . Solving $\nabla \hat{f} = \mathbf{0}$ for the *fitted* quadratic gives a stationary point that differs from (2, 2) by 1.0×10^{-14} in x_1 and 5×10^{-15} in x_2 , and a Hessian equal to $\mathbf{H}$ of Eq. (15) to within 4×10^{-15} , reproducing the hand derivation of Eq. (16) from the surrogate itself.

D. Held-Out Validation, Cross-Validation and GP Hyperparameters

The Trid metrics are in Tables 4 and 5, and the parity plots in Fig. 17. All models beat the baseline (test RMSE 9.73, $R^2 = -0.26$). The polynomial is exact to 7.9×10^{-15} ; the GP reaches 2.9×10^{-5} (NRMSE 1.0×10^{-6}); and the RBF reaches 0.042 (NRMSE 1.5×10^{-3}) with a test MaxAE of 0.155. As for Rosenbrock, the RBF test RMSE underestimates the grid RMSE (0.156) by a factor of almost four, while CV (0.178) is close to it.

The GP length scales are 37.5 and 37.2 in physical units (Table 6), almost identical and about 4.7 times the box width. Equal length scales are expected: the Trid function is symmetric under $x_1 \leftrightarrow x_2$ and so is the design box. Their size says that the response is low-order and smooth relative to the domain, so the GP is effectively modeling a

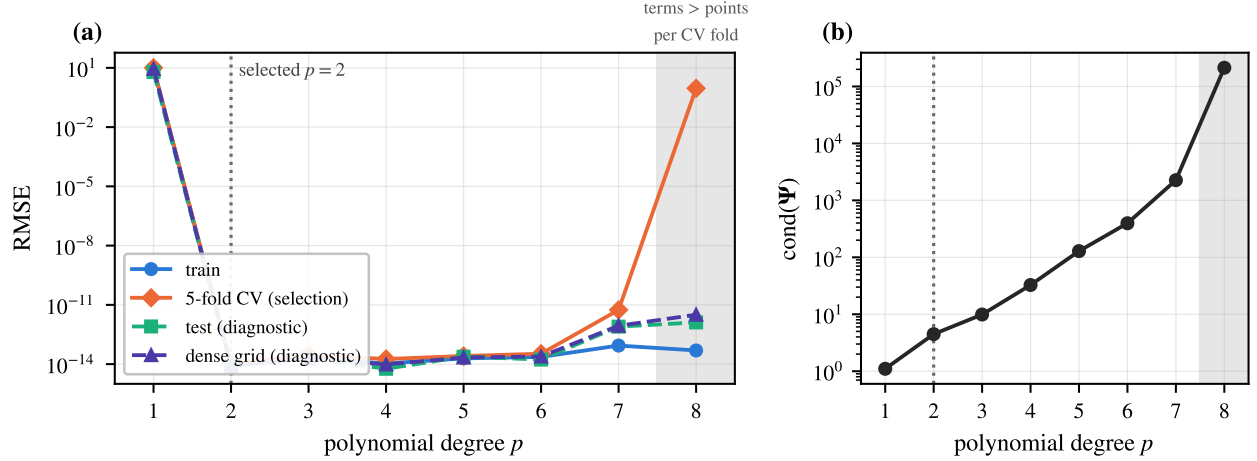


Fig. 16 Trid: polynomial complexity study. (a) Training, five-fold CV (selection) and test and dense-grid (diagnostic) RMSE against degree; the dotted line marks the selected $p = 2$ and the shaded band marks degrees with more coefficients than points per CV fold. (b) Condition number of the standardized design matrix.

polynomial-like trend. As for Rosenbrock, the signal variance sits at its upper bound and the noise variance near its lower bound (1.9×10^{-12}); here 8 of the 9 optimizer runs ended abnormally. Widening the bound gave lower likelihoods (260.1 and 254.6 against 302.6), which again reveals other local optima rather than better models, and worse CV errors (Table 6), so the 10^4 bound was kept for the reason given in Sec. III.F.

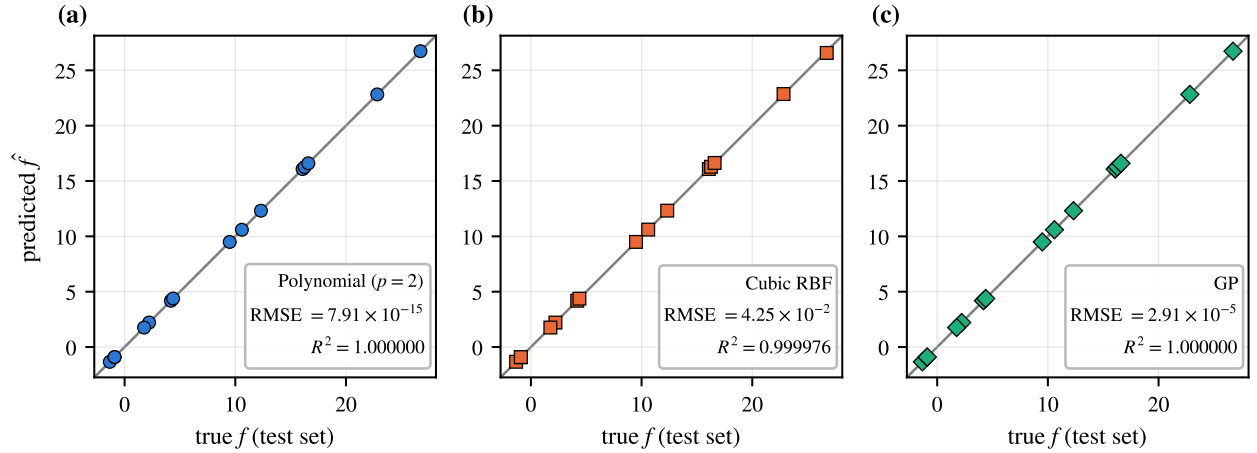


Fig. 17 Trid: parity plots on the 15 held-out test points for (a) the degree-2 polynomial, (b) the cubic RBF and (c) the GP, with test RMSE and R^2 in the insets.

E. Side-by-Side Visual Comparison

The side-by-side comparisons are in Figs. 18 to 20, and the 3-D pair of the best surrogate in Fig. 21. The polynomial error is round-off (2.8×10^{-14}). The GP error (Fig. 19) is below 10^{-4} in the interior and peaks at 9.6×10^{-4} at the edges and corners. The RBF error (Fig. 20) is the clearest failure pattern of the whole study: it is small inside the

sampled region (the learning curve of Fig. 23 gives a central-box RMSE of about 0.03 at $N = 45$) and grows sharply toward the boundary, reaching 2.72 in the corner $(-4, 4)$. The Trid function has constant curvature everywhere; the cubic RBF with a linear tail cannot represent a quadratic exactly, and near the boundary, where no samples lie beyond, it relaxes toward its linear tail and loses the curvature [19]. The error is therefore largest precisely where the true surface is steepest and the data give no support from outside.

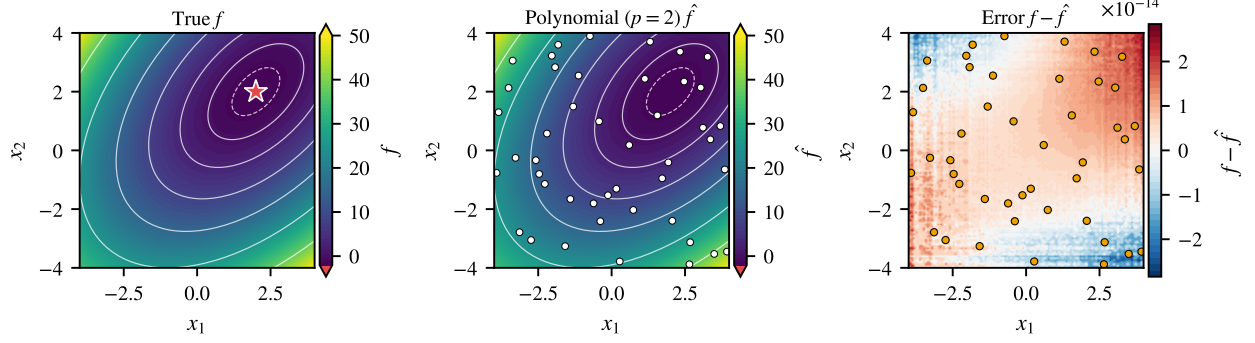


Fig. 18 Trid, best surrogate by CV (degree-2 polynomial): true f , surrogate $\hat{f}$ with identical color limits and training points, and error $f - \hat{f}$ on a diverging scale centered at zero with training points (maximum $|e| = 2.8 \times 10^{-14}$, round-off).

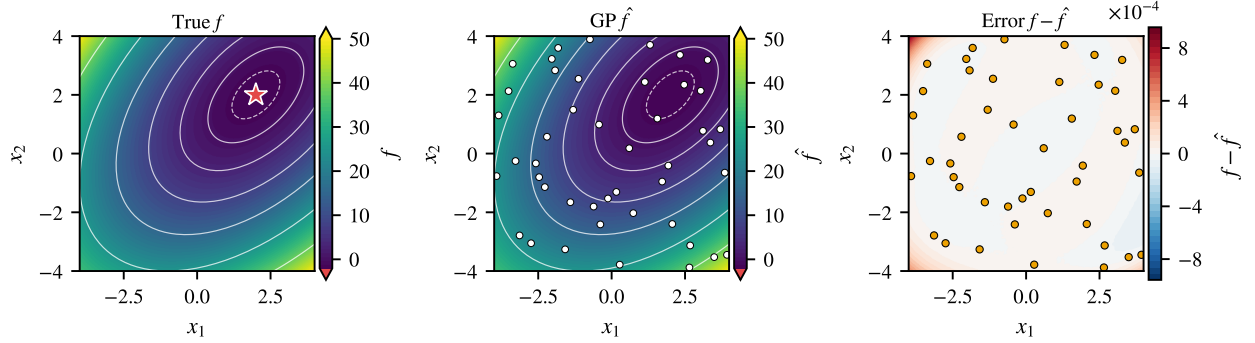


Fig. 19 Trid, GP surrogate: true f , surrogate $\hat{f}$ with identical color limits, and error $f - \hat{f}$ (diverging scale centered at zero, maximum $|e| = 9.6 \times 10^{-4}$), with training points overlaid.

F. Uncertainty Quantification

For the Trid GP (Fig. 22), $\hat{\sigma}$ is again smallest near the samples and largest at the edges and corners. Every grid point satisfies $|f - \hat{f}| \leq 2\hat{\sigma}$, so the uncertainty band is conservative. The Pearson correlation is 0.68, but the Spearman rank correlation is only 0.27. The reason is visible in panel (b): in the interior the true error is at the level of 10^{-5} to 10^{-8} and forms thin curved bands where $f - \hat{f}$ changes sign, a structure that has nothing to do with distance from the data. $\hat{\sigma}$ tracks the large errors at the edges (hence the reasonable Pearson value), but in the interior it is itself at its numerical floor: its median over the central box, 7.5×10^{-5} , is of the same order as the floor $\sqrt{\alpha + \sigma_n^2} \text{std}(y_{\text{train}}) = 1.2 \times 10^{-4}$ and as its value at the training points, 1.1×10^{-4} . Both $\hat{\sigma}$ and the true error are therefore dominated by numerical effects

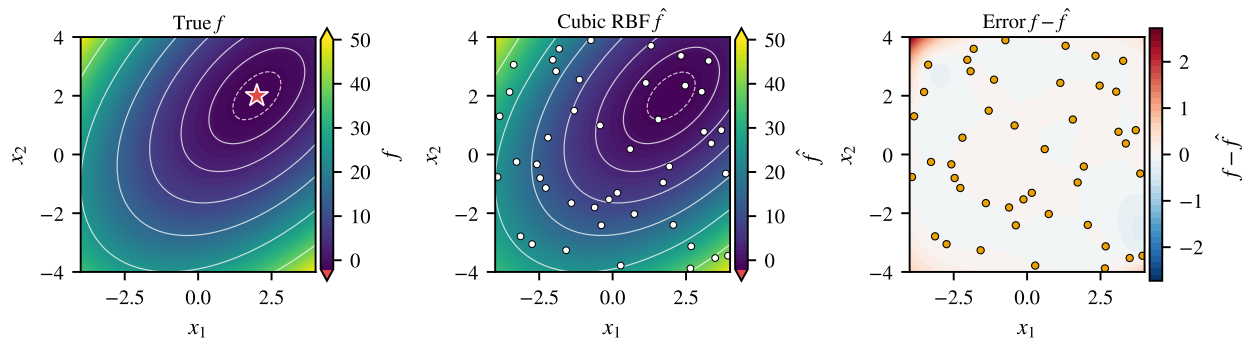


Fig. 20 Trid, cubic RBF surrogate: true f , surrogate $\hat{f}$ with identical color limits, and error $f - \hat{f}$ (diverging scale centered at zero, maximum $|e| = 2.72$ in the corner $(-4, 4)$), with training points overlaid. The error is confined to a band along the boundary.

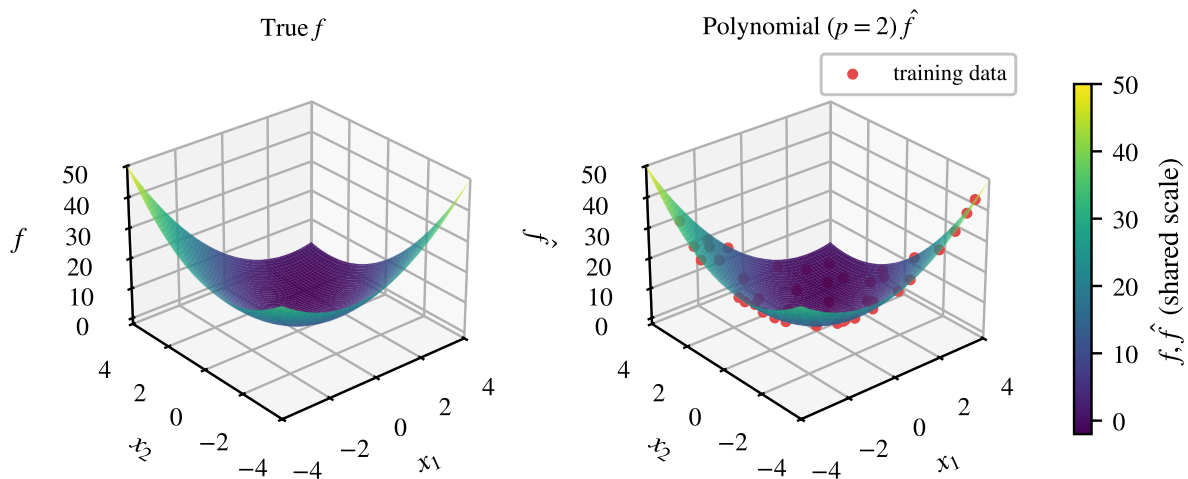


Fig. 21 Trid: 3-D surfaces of the true function and of the degree-2 polynomial surrogate with identical axis limits, color limits and viewing angle; red dots are the training data.

there, and one cannot rank the other. A rerun of the same script with the same package versions on a Linux machine confirms this: every other result in this report agrees within 1%, but the Trid correlations change to 0.20 (Spearman) and 0.65 (Pearson), because they are computed from quantities at round-off level. This is a clear illustration that $\hat{\sigma}$ is a statement about the model’s assumptions and the sample geometry, not a measurement of the actual error.

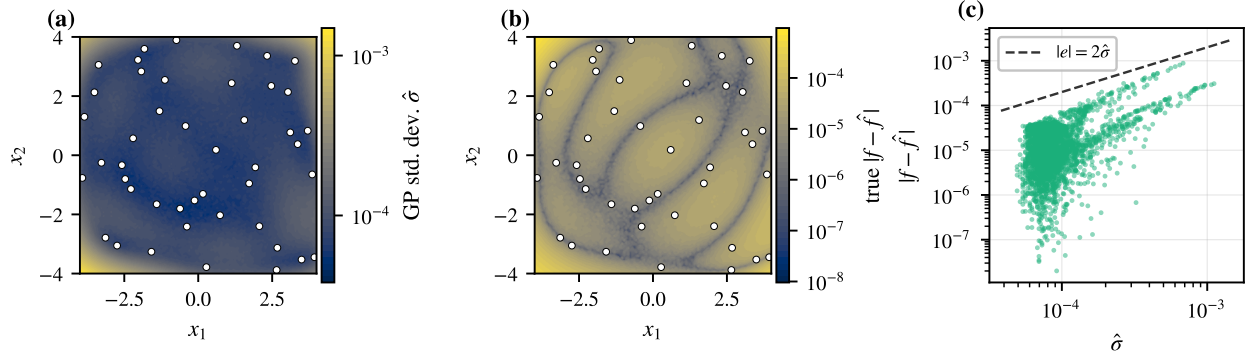


Fig. 22 Trid, GP uncertainty. (a) Predicted standard deviation $\hat{\sigma}$ and (b) true absolute error $|f - \hat{f}|$, both on log color scales with the training points. (c) $|f - \hat{f}|$ against $\hat{\sigma}$ for 3000 random grid points; every point lies below the line $|e| = 2\hat{\sigma}$.

G. Effect of Sample Size

The Trid learning curves (Fig. 23) separate interior from edge behavior. In the central box the RBF error falls steadily with N , from 0.235 to 0.0094, as a converging interpolant should. Over the full grid, however, its median RMSE stops falling after $N = 45$ (0.091, then 0.110 at $N = 65$ and 0.177 at $N = 90$). With five draws per size the ranges overlap (0.069 to 0.218 at $N = 45$, 0.117 to 0.265 at $N = 90$), so the rise itself is not significant, but the stagnation against a falling core error is clear. The median condition number of the saddle-point system grows from 1.0×10^4 at $N = 20$ to 2.2×10^6 at $N = 90$, far from causing round-off trouble in double precision, so conditioning is not the explanation. A plausible one is the boundary behavior of polyharmonic splines [19]: added interior points improve the interior but do little for the loss of curvature in the edge band, which dominates the full-grid error. The GP shows the same pattern at a much lower level: its full-grid median rises slightly (from 4.8×10^{-5} at $N = 45$ to 7.0×10^{-5} at $N = 90$) while its core error settles at about 1.7×10^{-5} . That floor is most likely set by the near-degenerate kernel (very long length scales and a noise variance near 10^{-12}), not by the amount of data. The polynomial is at round-off for every N .

H. Surrogate-Based Optimization and Verification

All three surrogates lead every one of the 20 starts to a single minimum (Table 7 and Fig. 24). The polynomial minimum lies 1.0×10^{-8} from $(2, 2)$, a distance set by the Nelder–Mead tolerance, with a true value of -2.000000 . The GP minimum is 0.00057 from $(2, 2)$ and its true value, -1.9999998 , agrees with the prediction to 2.7×10^{-7} . The RBF minimum is 0.0084 from $(2, 2)$, and it predicts $\hat{f} = -2.0123$, a value *lower* than the true global minimum -2 :

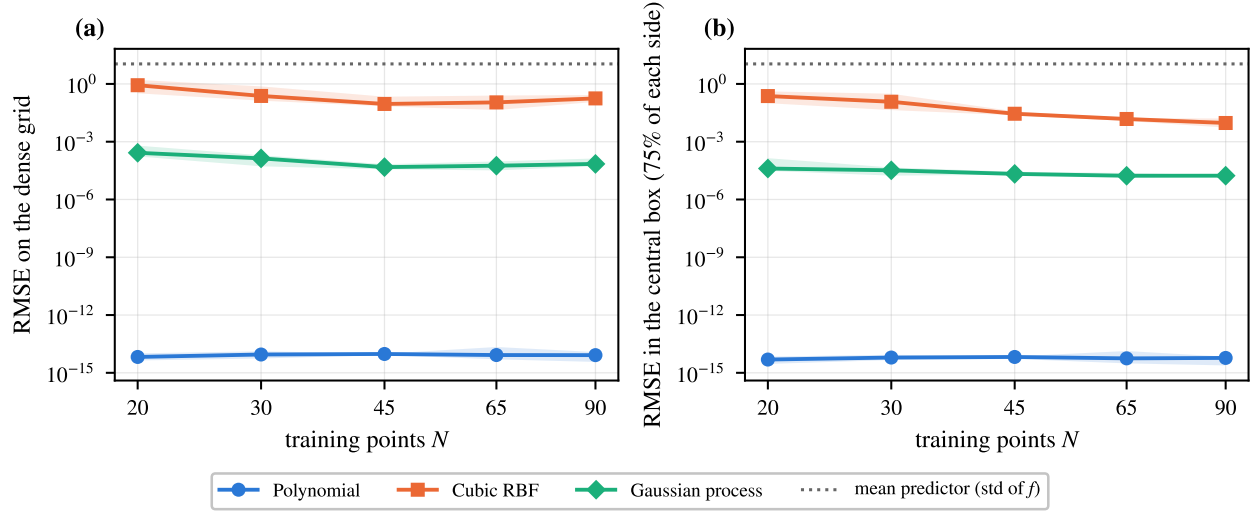


Fig. 23 Trid: learning curves on logarithmic axes. (a) RMSE over the full grid. (b) RMSE over the central box whose sides are 75% of the domain sides. Lines are medians over five LHS designs per size and bands their range; the dotted line is the error of predicting the mean (standard deviation of f , 10.9). Beyond $N = 45$ the full-grid RBF and GP errors stop falling, while their core errors do not rise.

the surrogate promises a design better than any design that exists. The true value at its point, -1.99996 , is still within 3.6×10^{-5} of the optimum, so for this convex function the positional error is harmless, but the predicted performance is optimistic. The multi-start search on the true function returns $(2, 2)$ and $f = -2$, confirming the hand derivation of Eq. (16) numerically. The analytical solution, the stationary point of the fitted quadratic and the numerical searches all agree.

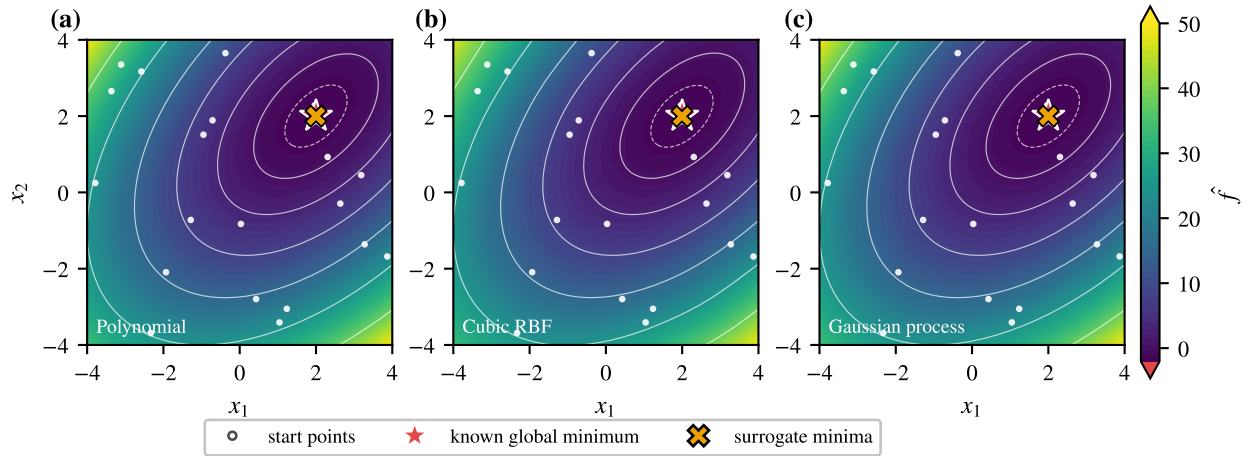


Fig. 24 Trid: multi-start optimization on (a) the degree-2 polynomial, (b) the cubic RBF and (c) the GP, drawn over each surrogate with the color limits of the true function; white dots are the 20 start points, crosses the surrogate minima and the star the known global minimum at $(2, 2)$.

I. Extrapolation

Trained on $[-2, 2]^2$, which has the minimum $(2, 2)$ at its corner, the GP error rises from 9.9×10^{-6} inside to 2.8×10^{-3} outside, a factor of 287 (Table 8 and Fig. 25). The ratio is smaller than for Rosenbrock because the Trid function keeps the same quadratic shape everywhere, so the long-length-scale GP continues the trend reasonably well. The RBF error grows by a factor of 70, to 19 at the corners, because its linear tail cannot follow the quadratic growth. The polynomial again extrapolates exactly, for the same reason as in Part A.

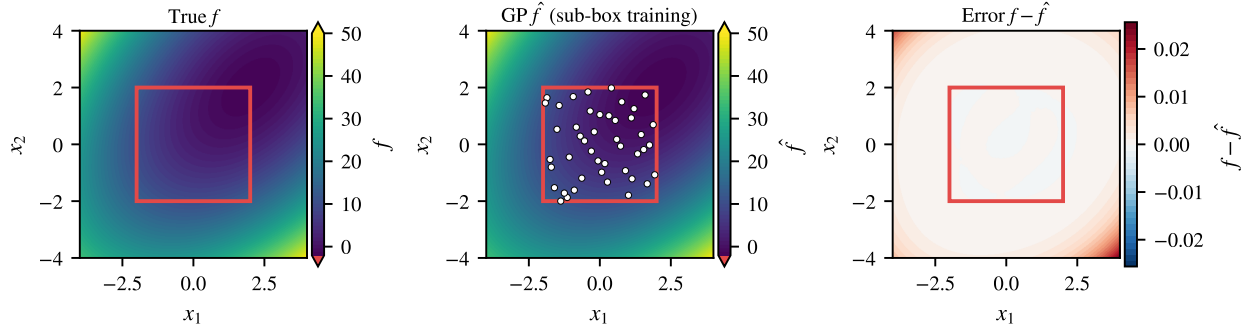


Fig. 25 Trid, extrapolation of a GP trained only inside the red box $[-2, 2]^2$ (white dots): true f , GP prediction with the color limits of the true function, and error $f - \hat{f}$, which grows toward the corners of the full domain.

J. Evaluation Budget

Table 9 separates the main DOE budget from the extra evaluations made only because the truth is cheap. Only the 60 DOE runs and the verification runs would exist in a real project; the grid, learning-curve, reduced-box and true-function multi-start evaluations are teaching diagnostics and were never used for fitting or selection.

Table 9 Accounting of true-function evaluations. Only the first two rows would be spent in a real study.

Purpose	Rosenbrock	Trid
Main DOE (45 training + 15 test)	60	60
Verification of surrogate optima	6	3
Dense truth grid (diagnostic)	14 400	14 400
Learning curves (5 sizes $\times$ 5 draws)	1250	1250
Reduced-box extrapolation training	45	45
Multi-start check on the true function	4424	3315

V. Discussion

A. Comparing the Two Functions

Both functions are exact polynomials, so the polynomial surrogate has zero bias and wins every comparison by nine or more orders of magnitude. That is not a fair fight, and it is not the main lesson. In a real project the true form

is unknown. What the data *do* show is a collapse of the CV error by 14 to 15 orders of magnitude at one particular degree, followed by a flat plateau at round-off. With noiseless simulation data such a collapse is strong evidence that the response is an exact low-degree polynomial; with noisy or non-polynomial data it would not occur, and the CV curve would instead have a shallow minimum.

The two functions differ sharply in difficulty for the non-polynomial surrogates. On a common scale (NRMSE, Table 4), the RBF is 12 times worse on Rosenbrock (1.9×10^{-2}) than on Trid (1.5×10^{-3}), and the GP about 7 times worse. The Rosenbrock function has a dynamic range of 3609 concentrated in the corners, quartic growth that a linear-tail RBF cannot follow, and a curved valley that axis-aligned length scales cannot describe. The Trid function has constant curvature, a range of 52 and a symmetry that the GP recovers as two equal length scales. The practical consequence is visible in the optimization: the same RBF that places the Trid minimum within 0.008 creates a false, negative minimum in the Rosenbrock valley.

B. Catalog Prompts

Rosenbrock. The GP length scales, 3.11 for x_1 and 27.4 for x_2 , correctly rank x_1 as the more influential variable. They cannot describe the valley itself, whose direction rotates across the domain. The “easy to find the valley, hard to find the minimum” character appears directly in Table 7: the GP optimum is displaced by 0.013 almost exactly along the valley tangent, at a true cost of only 4.5×10^{-5} , because the function is nearly flat in that direction. A tiny error in f coexists with a visible error in $\mathbf{x}$.

Trid. Degree 2 is exact, and CV selects it from the data. The minimum derived by hand, $(2, 2)$ with $f = -2$ from the 2×2 system of Eq. (15), is reproduced by the stationary point of the fitted quadratic to 10^{-14} , by the multi-start searches on the polynomial and the GP (within 10^{-8} and 6×10^{-4}), approximately by the RBF (within 0.008), and by the same search on the true function. The positive-definite Hessian, with eigenvalues 1 and 3, proves that this stationary point is the unique global minimum, which no numerical search alone can prove.

C. Connection to Theory

The results follow the bias–variance picture of Sec. II.E. Degrees below the true one fail through bias, with train, CV and test errors high and close together. Degrees above it show no variance-driven over-fitting, because the data are noiseless; they show numerical growth through the condition number, and under-determination when the CV folds have fewer points than coefficients. The RBF results match the known boundary behavior of polyharmonic splines [19]: interior accuracy converges with N , while the edge error stagnates. The GP results match the stationary-kernel picture [11]: accuracy is excellent where the data constrain the model, uncertainty grows away from the data, and the kernel drifts to its bounds when asked to represent a global polynomial trend.

D. Where Each Model Can Be Trusted

Table 10 summarizes the engineering judgment. The polynomial is trusted everywhere in both boxes, but that trust rests on the CV evidence of exact form. The GP is the model to use when the form is unknown: its interior accuracy is excellent, its optimum was verified within 0.013 (Rosenbrock) and 0.0006 (Trid), and $\hat{\sigma}$ points to the corners as its weakest region, although for Rosenbrock the $2\hat{\sigma}$ band underestimates the largest errors there. The RBF is acceptable for Trid trends and optimization in the interior, but not for Rosenbrock optimization, where it produces a spurious minimum with a physically impossible negative value. No model should be used outside its training box.

Table 10 Engineering judgment: which model to trust, for what purpose, and inside which bounds.

Function	Model	Recommended use and bounds
Rosenbrock	Polynomial, $p = 4$	Any purpose on $[-2, 2]^2$; exact form confirmed by the CV collapse.
	GP	Global trends and locating the optimum inside $[-2, 2]^2$ (verified $d = 0.013$); treat the corner regions, especially near $(-2, -2)$, with caution (error up to 0.70).
	Cubic RBF	Coarse trends only (test NRMSE 1.9%); not for optimization, because it creates spurious minima with $\hat{f} < 0$ in the valley.
Trid	Polynomial, $p = 2$	Any purpose on $[-4, 4]^2$; reproduces the analytical minimum and Hessian to round-off.
	GP	Any purpose on $[-4, 4]^2$ (NRMSE 10^{-6} , optimum within 6×10^{-4}).
	Cubic RBF	Interior of the box ($ x_j \lesssim 3$) and locating the optimum ($d = 0.008$); its predicted optimal value is optimistic by 0.012, and errors reach 2.7 at the corners.

E. Limitations

Every result comes from one DOE and one split (seed 7); the learning curves show that single-seed errors can vary by an order of magnitude at small N , so the reported numbers should be read as one realization rather than as exact accuracies. Both functions are smooth polynomials; the conclusions about the polynomial surrogate would not transfer to non-polynomial responses. The GP uses a stationary squared-exponential kernel whose signal variance sits at its bound; universal Kriging or a different kernel was not explored. Finally, the design is one-shot: in a real study the verified optimum candidates would be added to the DOE and the surrogate refitted, for example with expected-improvement infill [12].

VI. Conclusions

A complete surrogate study was carried out on the Rosenbrock and Trid functions with a budget of 60 evaluations from a maximin Latin hypercube design, whose minimum pairwise distance is almost twice that of random sampling at the same cost. The main findings are:

- 1) Training-only cross-validation recovered the true polynomial degrees (4 and 2) from the data, and the fitted coefficients matched the exact expansions to within 4×10^{-13} . Degrees above the true one showed no noise-driven over-fitting, but they did show conditioning growth and under-determination inside the CV folds.
- 2) The GP was the best model that does not assume the true form, with test NRMSE of 7.8×10^{-6} (Rosenbrock) and 1.0×10^{-6} (Trid). Its uncertainty $\hat{\sigma}$ correlated with the true error (Pearson 0.70 and 0.68) and bounded it within $2\hat{\sigma}$ at 98% and 100% of grid points. For Rosenbrock, however, the points outside the band include the worst error, and for Trid $\hat{\sigma}$ could not rank errors near the numerical floor.
- 3) The cubic RBF failed systematically at the domain boundaries. For Trid its full-grid error stopped improving beyond $N = 45$ while its interior error kept falling. For Rosenbrock it produced a spurious negative minimum, and only verification with the true function exposed it.
- 4) Surrogate optima were verified with one true evaluation each. The GP located the Rosenbrock minimum within 0.013, displaced along the flat valley, and the Trid minimum within 6×10^{-4} . The hand-derived Trid minimum $f = -2$ at (2, 2) was confirmed analytically, from the fitted quadratic, and numerically.
- 5) Outside a reduced training box, the RBF and GP errors grew by factors of 70 to 2300. A surrogate is a statement about the region it has seen, and must not be used outside it.

A cheap model can guide a design decision reliably, but only inside its sampled box, only after its prediction has been checked against the expensive model, and only when the engineer can show where it fails.

Acknowledgments

The workflow, the Latin hypercube and cubic RBF implementations, and the GP set-up are adapted from the AE 541 reference notebook on the Himmelblau function [6]; all functions, domains, settings, results and interpretations are the author’s own for the Rosenbrock and Trid functions. In line with Sec. 7 of the assignment brief, an AI assistant (Claude, Anthropic) was used to help structure the Python script and the $\text{\LaTeX}$ report, to review figure layout, and to improve the clarity of the English. Every equation, number and statement was checked by the author against the output of the submitted script, and the author takes full responsibility for the content.

References

- [1] Queipo, N. V., Haftka, R. T., Shyy, W., Goel, T., Vaidyanathan, R., and Tucker, P. K., “Surrogate-Based Analysis and Optimization,” *Progress in Aerospace Sciences*, Vol. 41, No. 1, 2005, pp. 1–28. <https://doi.org/10.1016/j.paerosci.2005.02.001>.
- [2] Forrester, A. I. J., Sóbester, A., and Keane, A. J., *Engineering Design via Surrogate Modelling: A Practical Guide*, Wiley, Chichester, UK, 2008. <https://doi.org/10.1002/9780470770801>.
- [3] AE 541 Course Team, “Assignment 1: Surrogate Model Development for Two-Variable Test Functions,” AE 541 Digital Twin in Modern Engineering, King Fahd University of Petroleum and Minerals, 2026.

- [4] Rosenbrock, H. H., “An Automatic Method for Finding the Greatest or Least Value of a Function,” *The Computer Journal*, Vol. 3, No. 3, 1960, pp. 175–184. <https://doi.org/10.1093/comjnl/3.3.175>.
- [5] Surjanovic, S., and Bingham, D., “Virtual Library of Simulation Experiments: Test Functions and Datasets,” Simon Fraser University, <https://www.sfu.ca/~ssurjano/>, 2013. Accessed September 2026.
- [6] AE 541 Course Team, “From an Expensive Function to a Trustworthy Surrogate: A Complete, Step-by-Step Study Using the Himmelblau Function,” Jupyter notebook `himmelblau_surrogate_tutorial.ipynb`, AE 541 Digital Twin in Modern Engineering, King Fahd University of Petroleum and Minerals, 2026.
- [7] McKay, M. D., Beckman, R. J., and Conover, W. J., “A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code,” *Technometrics*, Vol. 21, No. 2, 1979, pp. 239–245. <https://doi.org/10.2307/1268522>.
- [8] Johnson, M. E., Moore, L. M., and Ylvisaker, D., “Minimax and Maximin Distance Designs,” *Journal of Statistical Planning and Inference*, Vol. 26, No. 2, 1990, pp. 131–148. [https://doi.org/10.1016/0378-3758\(90\)90122-B](https://doi.org/10.1016/0378-3758(90)90122-B).
- [9] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, Vol. 12, 2011, pp. 2825–2830.
- [10] Buhmann, M. D., *Radial Basis Functions: Theory and Implementations*, Cambridge University Press, Cambridge, UK, 2003. <https://doi.org/10.1017/CBO9780511543241>.
- [11] Rasmussen, C. E., and Williams, C. K. I., *Gaussian Processes for Machine Learning*, MIT Press, Cambridge, MA, 2006.
- [12] Jones, D. R., Schonlau, M., and Welch, W. J., “Efficient Global Optimization of Expensive Black-Box Functions,” *Journal of Global Optimization*, Vol. 13, No. 4, 1998, pp. 455–492. <https://doi.org/10.1023/A:1008306431147>.
- [13] Hastie, T., Tibshirani, R., and Friedman, J., *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed., Springer, New York, 2009. <https://doi.org/10.1007/978-0-387-84858-7>.
- [14] Nelder, J. A., and Mead, R., “A Simplex Method for Function Minimization,” *The Computer Journal*, Vol. 7, No. 4, 1965, pp. 308–313. <https://doi.org/10.1093/comjnl/7.4.308>.
- [15] Harris, C. R., Millman, K. J., van der Walt, S. J., et al., “Array Programming with NumPy,” *Nature*, Vol. 585, 2020, pp. 357–362. <https://doi.org/10.1038/s41586-020-2649-2>.
- [16] Virtanen, P., Gommers, R., Oliphant, T. E., et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, Vol. 17, 2020, pp. 261–272. <https://doi.org/10.1038/s41592-019-0686-2>.
- [17] Hunter, J. D., “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, Vol. 9, No. 3, 2007, pp. 90–95. <https://doi.org/10.1109/MCSE.2007.55>.

- [18] McKinney, W., “Data Structures for Statistical Computing in Python,” *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- [19] Fornberg, B., Driscoll, T. A., Wright, G., and Charles, R., “Observations on the Behavior of Radial Basis Function Approximations near Boundaries,” *Computers & Mathematics with Applications*, Vol. 43, No. 3–5, 2002, pp. 473–490. [https://doi.org/10.1016/S0898-1221\(01\)00299-1](https://doi.org/10.1016/S0898-1221(01)00299-1).

A. Complete Python Code

The listing below is the complete file 202510630_Bahaa_Karawia.py; running it reproduces every figure, table and number in this report.

```
1 """
2 AE 541 - Digital Twin in Modern Engineering
3 Assignment 1: Surrogate model development for two-variable test functions
4
5 Author : Bahaa Karawia (KFUPM ID 202510630)
6 Part A : Rosenbrock function (common)
7 Part B : Trid function, 2 variables (assigned)
8
9 The workflow follows the course reference notebook
10 (chimmelblau_surrogate_tutorial.ipynb); the LHS, cubic RBF and GP set-up are
11 adapted from it with acknowledgment.
12
13 Run: python 202510630_Bahaa_Karawia.py
14 Needs: numpy, scipy, matplotlib, scikit-learn, pandas (versions in requirements.txt).
15 Repository: https://github.com/BahaaIbrahimK/AE541-Assignment1-Surrogate-Modeling
16 Every figure and table in the report is written to ./outputs next to this file.
17 """
18
19 import json
20 import platform
21 import warnings
22 from pathlib import Path
23
24 import matplotlib
25 matplotlib.use("Agg") # write files only, no GUI window
26 import matplotlib.pyplot as plt
27 import numpy as np
28 import pandas as pd
29 import scipy
30 import sklearn
31 from matplotlib.patches import Rectangle
32 from matplotlib.colors import LogNorm
33 from matplotlib.ticker import LogLocator, MaxNLocator, ScalarFormatter
34 from scipy.optimize import minimize
35 from scipy.spatial.distance import cdist, pdist
36 from scipy.stats import spearmanr
37 from sklearn.base import BaseEstimator, RegressorMixin
38 from sklearn.exceptions import ConvergenceWarning
39 from sklearn.gaussian_process import GaussianProcessRegressor
40 from sklearn.gaussian_process.kernels import RBF, ConstantKernel, WhiteKernel
41 from sklearn.linear_model import LinearRegression
42 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
43 from sklearn.model_selection import KFold, cross_val_predict, train_test_split
44 from sklearn.pipeline import make_pipeline
45 from sklearn.preprocessing import PolynomialFeatures, StandardScaler
46
47
48 # ===== CONFIGURATION =====
49 SEED = 7 # master seed; every random draw derives from it
50 SEED_DOE_STUDY = SEED + 1 # repeated designs for the random-vs-LHS comparison
51 SEED_STARTS = SEED + 2 # start points of the multi-start optimisation
52 SEED_SUB_BOX = SEED + 3 # design inside the reduced (extrapolation) box
53 SEED_STRIDE = 1000 # learning-curve draw r at size N uses SEED + 1000 r + N
54 N_SAMPLES = 60 # main DOE budget = training + held-out test
55 TEST_FRACTION = 0.25 # 45 training / 15 test points
56 CV_FOLDS = 5 # K-fold CV, applied to the training set only
57 POLY_DEGREES = range(1, 9) # degree sweep for the polynomial surrogate
58 PARSIMONY_TOL = 0.05 # degrees within 5 % of the best CV error count as tied
59 ROUNDOFF_REL = 1e-9 # CV RMSE below this x std(y_train) counts as exact
60 MAXIMIN_CANDIDATES = 100 # LHS candidates screened for the maximin design
61 DOE_REALISATIONS = 500 # repeated designs used to compare the DOE criteria
62 GRID_N = 120 # dense truth grid, GRID_N x GRID_N
63 GP_RESTARTS = 8 # restarts of the marginal-likelihood optimiser
64 GP_SIGNAL_MIN = 1e-3 # bounds on sigma_f^2 (outputs are normalised)
65 GP_SIGNAL_MAX = 1e4
66 GP_SIGNAL_TRIALS = [1e4, 1e6, 1e8] # upper bounds tried when sigma_f^2 hits its limit
67 GP_LENGTH_BOUNDS = (1e-2, 1e3) # length scales, in standardised input units
68 GP_NOISE_BOUNDS = (1e-12, 1e-2) # sigma_n^2; tiny because the data are noiseless
69 GP_INIT = {"signal": 1.0, "length": [1.0, 1.0], "noise": 1e-8} # optimiser start
70 GP_ALPHA = 1e-10 # diagonal jitter for numerical stability (sklearn default);
71 # together with sigma_n^2 it sets a floor under sigma_hat
72 GP_BOUND_TOL = 1e-2 # |log theta - log bound| below this counts as "at bound"
73 CORE_FRAC = 0.75 # central fraction of the box used for "core" errors
```

```

74 LEARNING_N = [20, 30, 45, 65, 90] # training sizes for the learning curve
75 LEARNING_REPEATS = 5 # independent LHS draws per training size
76 N_STARTS = 20 # multi-start local searches on each surrogate
77 NM_XATOL, NM_FATOL = 1e-9, 1e-12 # Nelder-Mead stopping tolerances
78 NM_MAXITER = 5000
79 CLUSTER_FRAC = 0.02 # minima closer than this x box width are merged
80 BOUNDARY_TOL = 1e-6 # an optimum this close to a bound is flagged
81 SCATTER_POINTS = 3000 # grid points shown in the sigma-vs-error scatter
82 UNDERSHOOT_TOL = 1e-3 # surrogate values below min(f) by more than this x range
83 # of f are drawn in red on the field maps
84 OUTPUT_DIR = Path(__file__).resolve().parent / "outputs"
85
86
87 def rosenbrock(x1, x2):
88     return 100.0 * (x2 - x1**2) ** 2 + (1.0 - x1) ** 2
89
90
91 def trid(x1, x2):
92     return (x1 - 1.0) ** 2 + (x2 - 1.0) ** 2 - x1 * x2
93
94
95 FUNCTIONS = {
96     "rosenbrock": {
97         "name": "Rosenbrock",
98         "func": rosenbrock,
99         "bounds": np.array([[ -2.0, 2.0], [ -2.0, 2.0]]),
100         "minima": np.array([[1.0, 1.0]]),
101         "f_min": 0.0,
102         "sub_box": np.array([[ -1.0, 1.0], [ -1.0, 1.0]]),
103         # expanded form:  $100x_1^4 - 200x_1^2x_2 + 100x_2^2 + x_1^2 - 2x_1 + 1$ 
104         "exact_coefs": {"1": 1.0, "x1": -2.0, "x1^2": 1.0, "x2^2": 100.0,
105                        "x1^2 x2": -200.0, "x1^4": 100.0},
106         "line_levels": [0.5, 3, 15, 60, 250, 1000, 2500],
107         "view": (28, -118),
108     },
109     "trid": {
110         "name": "Trid",
111         "func": trid,
112         "bounds": np.array([[ -4.0, 4.0], [ -4.0, 4.0]]),
113         "minima": np.array([[2.0, 2.0]]),
114         "f_min": -2.0,
115         "sub_box": np.array([[ -2.0, 2.0], [ -2.0, 2.0]]),
116         # expanded form:  $x_1^2 + x_2^2 - x_1x_2 - 2x_1 - 2x_2 + 2$ 
117         "exact_coefs": {"1": 2.0, "x1": -2.0, "x2": -2.0, "x1^2": 1.0,
118                        "x1 x2": -1.0, "x2^2": 1.0},
119         "line_levels": [-1.5, 0, 3, 8, 15, 25, 38],
120         "view": (28, -130),
121     },
122 }
123
124 MODEL_NAMES = ["Polynomial", "Cubic RBF", "Gaussian process"]
125 MODEL_COLORS = {"Polynomial": "#2a78d6", "Cubic RBF": "#eb6834",
126                "Gaussian process": "#1baf7a"}
127 MODEL_MARKERS = {"Polynomial": "o", "Cubic RBF": "s", "Gaussian process": "D"}
128 # =====
129
130
131 def set_plot_style():
132     plt.rcParams.update({
133         "font.family": "serif",
134         "font.serif": ["STIXGeneral", "Times New Roman", "DejaVu Serif"],
135         "mathtext.fontset": "stix",
136         "font.size": 9,
137         "axes.labelsize": 9.5,
138         "xtick.labelsize": 8.5,
139         "ytick.labelsize": 8.5,
140         "legend.fontsize": 8,
141         "axes.linewidth": 0.8,
142         "xtick.direction": "out",
143         "ytick.direction": "out",
144         "lines.linewidth": 1.5,
145         "lines.markersize": 5,
146         "legend.framealpha": 0.92,
147         "legend.edgecolor": "0.75",
148         "savefig.dpi": 300,
149         "savefig.bbox": "tight",
150         "savefig.pad_inches": 0.03,
151     })
152
153
154 def save_figure(fig, folder, name):

```

```

155 folder.mkdir(parents=True, exist_ok=True)
156 fig.savefig(folder / f"{name}.pdf")
157 fig.savefig(folder / f"{name}.png")
158 plt.close(fig)
159
160
161 def panel_letter(ax, letter, three_d=False):
162     # letters sit just outside the top-left corner so they never cover data
163     kw = dict(fontsize=10, fontweight="bold")
164     if three_d:
165         ax.text2D(0.0, 0.97, f"({letter})", transform=ax.transAxes, **kw)
166     else:
167         ax.text(-0.02, 1.02, f"({letter})", transform=ax.transAxes,
168                va="bottom", ha="right", **kw)
169
170
171 def field_levels(Y):
172     """Contour levels for f and f_hat maps, plus a colour map that flags undershoot.
173
174     The lower limit sits just below min(f), so round-off never triggers the
175     flag, while a surrogate that predicts clearly impossible values shows up red.
176     """
177     lo, hi = Y.min(), Y.max()
178     cmap = plt.get_cmap("viridis").copy()
179     cmap.set_under("#e34948")
180     return np.linspace(lo - UNDERSHOOT_TOL * (hi - lo), hi, 61), cmap
181
182
183 def latex_sci(v, digits=2):
184     mantissa, exponent = f"{v:.{digits}e}".split("e")
185     return rf"{mantissa}\times 10^{{{int(exponent)}}}"
186
187
188 def sci_colorbar(fig, mappable, ax, label):
189     cb = fig.colorbar(mappable, ax=ax, pad=0.02, fraction=0.05)
190     lo, hi = mappable.norm.vmin, mappable.norm.vmax
191     # keep ticks inside the data range, otherwise extend="both" colour bars
192     # get a tick drawn on the arrow that collides with its neighbour
193     cb.set_ticks([t for t in MaxNLocator(6).tick_values(lo, hi) if lo <= t <= hi])
194     fmt = ScalarFormatter(useMathText=True)
195     fmt.set_powerlimits((-3, 4))
196     cb.formatter = fmt
197     cb.update_ticks()
198     cb.set_label(label)
199     return cb
200
201
202 # ----- design of experiments -----
203
204 def latin_hypercube(n, d, rng):
205     """Classic LHS in the unit cube: one point per bin on every axis."""
206     bins = np.column_stack([rng.permutation(n) for _ in range(d)])
207     return (bins + rng.random((n, d))) / n
208
209
210 def uniform_random(n, d, rng):
211     return rng.random((n, d))
212
213
214 def maximin_lhs(n, d, rng, n_candidates):
215     """Best of several LHS draws by the maximin criterion.
216
217     Generating a design costs no function evaluations, so screening candidates
218     improves the spread for free while keeping the LHS stratification.
219     """
220     best, best_score = None, -np.inf
221     for _ in range(n_candidates):
222         U = latin_hypercube(n, d, rng)
223         score = pdist(U).min()
224         if score > best_score:
225             best, best_score = U, score
226     return best
227
228
229 def to_box(U, bounds):
230     return bounds[:, 0] + U * (bounds[:, 1] - bounds[:, 0])
231
232
233 def min_distance(U):
234     return pdist(U).min()
235

```

```

236
237 def corner_gaps(X_train, bounds):
238     """Distance from each corner of the box to its nearest training point."""
239     corners = np.array([[bounds[0], i], bounds[1, j]] for i in (0, 1) for j in (0, 1)])
240     return {f"{c[0]:g}, {c[1]:g}": float(np.min(np.linalg.norm(X_train - c, axis=1)))
241             for c in corners}
242
243
244 def compare_designs(rng):
245     """Distribution of the maximin criterion over many realisations, in [0,1]^2."""
246     scores = {"Uniform random": [], "LHS": []}
247     for _ in range(DOE_REALISATIONS):
248         scores["Uniform random"].append(min_distance(uniform_random(N_SAMPLES, 2, rng)))
249         scores["LHS"].append(min_distance(latin_hypcube(N_SAMPLES, 2, rng)))
250     return {k: np.array(v) for k, v in scores.items()}
251
252
253 # ----- surrogates -----
254
255 class CubicRBF(RegressorMixin, BaseEstimator):
256     """Cubic RBF interpolant with a linear polynomial tail.
257
258      $f_{\text{hat}}(x) = \sum_i w_i ||x - x_i||^3 + c_0 + c_1 x_1 + c_2 x_2$ , with  $\sum w_i = 0$  and
259      $\sum w_i x_i = 0$  so that the weights carry no linear trend of their own.
260     """
261
262     def fit(self, X, y):
263         X, y = np.asarray(X, float), np.asarray(y, float)
264         n, d = X.shape
265         Phi = cdist(X, X) ** 3
266         P = np.hstack([np.ones((n, 1)), X])
267         A = np.block([[Phi, P], [P.T, np.zeros((d + 1, d + 1))]])
268         sol = np.linalg.solve(A, np.concatenate([y, np.zeros(d + 1)]))
269         self.centres_, self.weights_, self.tail_ = X, sol[:n], sol[n:]
270         self.condition_number_ = np.linalg.cond(A)
271         return self
272
273     def predict(self, X):
274         X = np.asarray(X, float)
275         P = np.hstack([np.ones((len(X), 1)), X])
276         return cdist(X, self.centres_) ** 3 @ self.weights_ + P @ self.tail_
277
278
279 def make_polynomial(degree):
280     # scaling sits inside the pipeline, so every fit (including each CV fold)
281     # learns mu and sigma from its own training rows only - no leakage
282     return make_pipeline(StandardScaler(),
283                          PolynomialFeatures(degree, include_bias=False),
284                          LinearRegression())
285
286
287 def make_rbf():
288     return make_pipeline(StandardScaler(), CubicRBF())
289
290
291 def make_gp(seed=SEED, signal_max=GP_SIGNAL_MAX):
292     kernel = (ConstantKernel(GP_INIT["signal"], (GP_SIGNAL_MIN, signal_max))
293              * RBF(length_scale=GP_INIT["length"], length_scale_bounds=GP_LENGTH_BOUNDS)
294              + WhiteKernel(GP_INIT["noise"], GP_NOISE_BOUNDS))
295     return make_pipeline(StandardScaler(),
296                          GaussianProcessRegressor(kernel=kernel, normalize_y=True, alpha=GP_ALPHA,
297                                                    n_restarts_optimizer=GP_RESTARTS,
298                                                    random_state=seed))
299
300
301 def make_model(name, degree):
302     if name == "Polynomial":
303         return make_polynomial(degree)
304     if name == "Cubic RBF":
305         return make_rbf()
306     return make_gp()
307
308
309 def n_terms(p):
310     return (p + 1) * (p + 2) // 2
311
312
313 def gp_bound_sensitivity(X_train, y_train):
314     """Refit the GP with wider sigma_f^2 bounds, judged by training CV only.
315
316     A wider bound still contains the old optimum, so a lower likelihood after

```

```

317 widening means the optimiser failed, not that the model got worse; the
318 abnormal terminations are counted to make that visible.
319 """
320 rows = []
321 for smax in GP_SIGNAL_TRIALS:
322     m = make_gp(signal_max=smax)
323     n_abnormal = fit_gp_counting_failures(m, X_train, y_train)
324     hp = gp_hyperparameters(m)
325     rows.append({"signal_max": smax,
326                 "signal_variance": hp["signal_variance_normalised"],
327                 "ell1_phys": hp["length_scale_phys"][0],
328                 "ell2_phys": hp["length_scale_phys"][1],
329                 "noise_variance": hp["noise_variance_normalised"],
330                 "log_marginal_likelihood": hp["log_marginal_likelihood"],
331                 "CV_RMSE": cv_rmse(make_gp(signal_max=smax), X_train, y_train),
332                 "abnormal_terminations": n_abnormal,
333                 "optimizer_runs": GP_RESTARTS + 1,
334                 "at_bounds": ";" . join(hp["parameters_at_bounds"])}))
335 return pd.DataFrame(rows)
336
337
338 def gp_hyperparameters(gp_pipeline):
339     """Fitted GP kernel parameters, with length scales in physical units too."""
340     gp = gp_pipeline[-1]
341     scaler = gp_pipeline[0]
342     k = gp.kernel_
343     ell_std = np.atleast_1d(k.k1.k2.length_scale)
344     at_bound = []
345     for hp, theta in zip(k.hyperparameters, k.theta):
346         lo, hi = np.log(hp.bounds[0])
347         if np.isclose(theta, lo, atol=GP_BOUND_TOL) or np.isclose(theta, hi, atol=GP_BOUND_TOL):
348             at_bound.append(hp.name)
349     return {
350         "signal_variance_normalised": float(k.k1.k1.constant_value),
351         "length_scale_std": ell_std.tolist(),
352         "length_scale_phys": (ell_std * scaler.scale_).tolist(),
353         "noise_variance_normalised": float(k.k2.noise_level),
354         "log_marginal_likelihood": float(gp.log_marginal_likelihood_value_),
355         "parameters_at_bounds": at_bound,
356     }
357
358
359 # ----- validation -----
360
361 def metrics(y_true, y_pred):
362     y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
363     rmse = float(np.sqrt(mean_squared_error(y_true, y_pred)))
364     y_range = y_true.max() - y_true.min()
365     return {"RMSE": rmse,
366             "MAE": float(mean_absolute_error(y_true, y_pred)),
367             "R2": float(r2_score(y_true, y_pred)),
368             "MaxAE": float(np.max(np.abs(y_true - y_pred))),
369             "NRMSE": rmse / y_range if y_range > 0 else float("nan")}
370
371
372 def cv_rmse(model, X, y):
373     """Pooled out-of-fold RMSE; cross_val_predict refits the whole pipeline per fold."""
374     folds = KFold(CV_FOLDS, shuffle=True, random_state=SEED)
375     with warnings.catch_warnings():
376         # GP bound warnings are checked explicitly on the final model instead
377         warnings.simplefilter("ignore", ConvergenceWarning)
378         pred = cross_val_predict(model, X, y, cv=folds)
379     return float(np.sqrt(mean_squared_error(y, pred)))
380
381
382 def design_matrix_condition(X_train, degree):
383     Z = StandardScaler().fit_transform(X_train)
384     return float(np.linalg.cond(PolynomialFeatures(degree).fit_transform(Z)))
385
386
387 def degree_sweep(X_train, y_train):
388     """Training-only quantities for each degree; nothing here sees the test set."""
389     rows = []
390     fold_size = int(len(X_train) * (CV_FOLDS - 1) / CV_FOLDS)
391     for p in POLY_DEGREES:
392         m = make_polynomial(p).fit(X_train, y_train)
393         rows.append({
394             "degree": p,
395             "n_terms": n_terms(p),
396             "points_per_fold": fold_size,
397             "cond_design": design_matrix_condition(X_train, p),

```

```

398         "train_RMSE": metrics(y_train, m.predict(X_train))["RMSE"],
399         "CV_RMSE": cv_rmse(make_polynomial(p), X_train, y_train),
400     })
401     return pd.DataFrame(rows)
402
403
404 def select_degree(sweep, y_train):
405     """Smallest degree whose CV error ties with the best one, or is at round-off.
406
407     With noiseless polynomial data every degree at or above the true one gives
408     round-off error, and the argmin among those is decided by noise. Preferring
409     the simplest adequate degree is the parsimony rule.
410     """
411     threshold = max((1 + PARSIMONY_TOL) * sweep["CV_RMSE"].min(),
412                     ROUNDOFF_REL * np.std(y_train))
413     return int(sweep.loc[sweep["CV_RMSE"] <= threshold, "degree"].min())
414
415
416 def add_sweep_diagnostics(sweep, X_train, y_train, X_test, y_test, X_grid, y_grid):
417     """Test and dense-grid error per degree, added only after the degree is locked."""
418     test, grid = [], []
419     for p in sweep["degree"]:
420         m = make_polynomial(p).fit(X_train, y_train)
421         test.append(metrics(y_test, m.predict(X_test))["RMSE"])
422         grid.append(metrics(y_grid, m.predict(X_grid))["RMSE"])
423     return sweep.assign(test_RMSE=test, grid_RMSE=grid)
424
425
426 def polynomial_readback(model, degree, X_grid):
427     """Coefficients of the fitted surrogate in the raw monomials of x.
428
429     An affine input scaling keeps the polynomial degree, so f_hat is an exact
430     degree-p polynomial in x and projecting it on raw monomials recovers them.
431     """
432     pf = PolynomialFeatures(degree)
433     Phi = pf.fit_transform(X_grid)
434     coef, *_ = np.linalg.lstsq(Phi, model.predict(X_grid), rcond=None)
435     return dict(zip(pf.get_feature_names_out(["x1", "x2"]), coef))
436
437
438 def quadratic_stationary_point(c):
439     """Solve grad f = 0 for f = c0 + a x1 + b x2 + A x1^2 + B x1 x2 + C x2^2."""
440     H = np.array([[2 * c["x1^2"], c["x1 x2"]], [c["x1 x2"], 2 * c["x2^2"]]])
441     g0 = np.array([c["x1"], c["x2"]])
442     return np.linalg.solve(H, -g0), H
443
444
445 # ----- optimisation -----
446
447 def multistart_minimise(objective, bounds, starts, cluster_tol):
448     """Bounded Nelder-Mead from every start, then merge nearby end points.
449
450     Nelder-Mead is derivative free, which avoids the finite-difference trap on
451     GP predictions noted in tutorial Sec. 15.2.
452     """
453     ends, n_eval = [], 0
454     for x0 in starts:
455         res = minimize(lambda x: float(objective(x[None, :])[0]), x0,
456                       method="Nelder-Mead", bounds=list(map(tuple, bounds)),
457                       options=dict(xatol=NM_XATOL, fatol=NM_FATOL, maxiter=NM_MAXITER))
458         ends.append((float(res.fun), res.x))
459         n_eval += res.nfev
460     ends.sort(key=lambda t: t[0])
461     clusters = []
462     for fval, x in ends:
463         for c in clusters:
464             if np.linalg.norm(x - c["x"]) <= cluster_tol:
465                 c["hits"] += 1
466                 break
467         else:
468             clusters.append({"f_hat": fval, "x": x, "hits": 1})
469     return clusters, n_eval
470
471
472 def on_boundary(x, bounds, tol=BOUNDARY_TOL):
473     return bool(np.any(np.abs(x - bounds[:, 0]) < tol) or np.any(np.abs(x - bounds[:, 1]) < tol))
474
475
476 # ----- figures -----
477
478 def plot_truth(spec, G, out):

```

```

479 X1, X2, Y = G["X1"], G["X2"], G["Y"]
480 fig = plt.figure(figsize=(7.2, 3.0))
481 ax = fig.add_subplot(1, 2, 1)
482 cs = ax.contourf(X1, X2, Y, levels=60, cmap="viridis")
483 cs.set_rasterized(True)
484 cl = ax.contour(X1, X2, Y, levels=spec["line_levels"], colors="white",
485               linewidths=0.6, alpha=0.8)
486 ax.clabel(cl, fontsize=7, fmt="%g")
487 if spec["name"] == "Rosenbrock":
488     x = np.linspace(-np.sqrt(2), np.sqrt(2), 200)
489     ax.plot(x, x**2, "--", color="#eb6834", lw=1.2, label=r"valley  $x_2=x_1^2$ ")
490 else:
491     t = np.linspace(-6, 6, 50)
492     ax.plot(2 + t, 2 + t, "--", color="#eb6834", lw=1.1, label="Hessian eigenvectors")
493     ax.plot(2 + t, 2 - t, "--", color="#eb6834", lw=1.1)
494     ax.set(xlim=spec["bounds"][0], ylim=spec["bounds"][1])
495     ax.plot(*spec["minima"].T, "*", ms=13, mfc="#e34948", mec="white", mew=0.8,
496           ls="none", label=f"global minimum,  $f={spec['f_min']:g}$ ")
497     ax.set(xlabel="$x_1$", ylabel="$x_2$", aspect="equal")
498     ax.legend(loc="lower right", fontsize=7.5)
499     sci_colorbar(fig, cs, ax, "$f(x_1, x_2)$")
500     panel_letter(ax, "a")
501
502 ax3 = fig.add_subplot(1, 2, 2, projection="3d")
503 ax3.plot_surface(X1, X2, Y, cmap="viridis", linewidth=0, antialiased=True,
504                rstride=2, cstride=2, rasterized=True)
505 fx, fy = spec["minima"][0]
506 ax3.scatter([fx], [fy], [spec["f_min"]], marker="*", s=90, c="#e34948",
507            edgecolors="white", depthshade=False, zorder=10)
508 ax3.set_xlabel("$x_1$", labelpad=-2)
509 ax3.set_ylabel("$x_2$", labelpad=-2)
510 ax3.set_zlabel("$f$", labelpad=0)
511 ax3.tick_params(pad=-1)
512 ax3.view_init(*spec["view"])
513 ax3.set_box_aspect(None, zoom=0.85)
514 panel_letter(ax3, "b", three_d=True)
515 fig.subplots_adjust(wspace=0.3)
516 save_figure(fig, out, "fig01-truth")
517
518
519 def plot_doe(spec, designs, scores, chosen_score, out):
520     fig, axes = plt.subplots(2, 2, figsize=(6.4, 6.0))
521     b = spec["bounds"]
522     pad = 0.07 * (b[0, 1] - b[0, 0])
523     for ax, (label, U), letter in zip(axes.flat[:3], designs.items(), "abc"):
524         X = to_box(U, b)
525         ax.plot(X[:, 0], X[:, 1], "o", ms=4.2, mfc="#2a78d6", mec="black", mew=0.4, ls="none")
526         # rug ticks along the edges show the one-dimensional projections
527         ax.plot(X[:, 0], np.full(len(X), b[1, 0] - 0.55 * pad), "|", color="#e34948", ms=6)
528         ax.plot(np.full(len(X), b[0, 0] - 0.55 * pad), X[:, 1], "_", color="#e34948", ms=6)
529         ax.set(xlim=(b[0, 0] - pad, b[0, 1] + 0.02 * pad),
530              ylim=(b[1, 0] - pad, b[1, 1] + 0.02 * pad),
531              xlabel="$x_1$", ylabel="$x_2$", aspect="equal")
532         panel_letter(ax, letter)
533     ax = axes[1, 1]
534     bins = np.linspace(0, max(s.max() for s in scores.values()) * 1.05, 40)
535     for (label, s), color in zip(scores.items(), ["#2a78d6", "#eb6834"]):
536         ax.hist(s, bins=bins, histtype="stepfilled", alpha=0.25, color=color)
537         ax.hist(s, bins=bins, histtype="step", lw=1.3, color=color,
538               label=f"{label} (median {np.median(s):.4f})")
539     ax.axvline(chosen_score, color="#4a3aa7", ls="--", lw=1.3, label="maximin LHS used")
540     ax.set(xlabel=r"minimum pairwise distance  $\phi_{\min}$  in  $[0, 1]^2$ ",
541           ylabel=f"count ({DOE_REALISATIONS} designs)")
542     ax.set_ylim(0, ax.get_ylim()[1] * 1.4) # headroom so the legend clears the bars
543     ax.set_xlim(0, 1.08 * max(chosen_score, bins[-1]))
544     ax.legend(fontsize=7.5, loc="upper left")
545     ax.set_box_aspect(1)
546     panel_letter(ax, "d")
547     fig.tight_layout()
548     save_figure(fig, out, "fig02-doe")
549
550
551 def plot_split(spec, G, X_train, X_test, out):
552     fig, ax = plt.subplots(figsize=(3.9, 3.4))
553     ax.contour(G["X1"], G["X2"], G["Y"], levels=spec["line_levels"], colors="0.55", linewidths=0.6)
554     ax.plot(X_train[:, 0], X_train[:, 1], "o", ms=5, mfc="#2a78d6", mec="black", mew=0.5,
555           ls="none", label=f"training ( $n={len(X_train)}$ ")
556     ax.plot(X_test[:, 0], X_test[:, 1], "s", ms=5.5, mfc="#eb6834", mec="black", mew=0.5,
557           ls="none", label=f"test ( $n={len(X_test)}$ ")
558     ax.plot(*spec["minima"].T, "*", ms=13, mfc="#e34948", mec="white", mew=0.8, ls="none",
559           label="global minimum")

```

```

560 ax.set(xlabel="$x_1$", ylabel="$x_2$", aspect="equal",
561        xlim=spec["bounds"][0], ylim=spec["bounds"][1])
562 ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.17), ncol=3, fontsize=7.5,
563          handletextpad=0.3, columnspacing=0.8)
564 save_figure(fig, out, "fig03_split")
565
566
567 def plot_degree_sweep(sweep, chosen, out):
568     fig, (ax, axc) = plt.subplots(1, 2, figsize=(7.0, 2.8), gridspec_kw={"width_ratios": [1.6, 1]})
569     p = sweep["degree"]
570     under = sweep.loc[sweep["n_terms"] > sweep["points_per_fold"], "degree"]
571     for a in (ax, axc):
572         if len(under):
573             a.axvspan(under.min() - 0.5, p.max() + 0.5, color="0.9", zorder=0)
574             a.axvline(chosen, color="0.35", ls=":", lw=1.2, zorder=1)
575             a.grid(alpha=0.3, lw=0.5)
576             a.set_xticks(list(p))
577     floor = 1e-16
578     series = [
579         ("train_RMSE", "train", "#2a78d6", "o", "-"),
580         ("CV_RMSE", f"{CV_FOLDS}-fold CV (selection)", "#eb6834", "D", "-"),
581         ("test_RMSE", "test (diagnostic)", "#1baf7a", "s", "--"),
582         ("grid_RMSE", "dense grid (diagnostic)", "#4a3aa7", "A", "--")]
583     for col, label, color, mk, ls in series:
584         ax.semilogy(p, np.maximum(sweep[col], floor), ls=ls, marker=mk, color=color,
585                     ms=4.5, label=label)
586     ax.set(xlabel="polynomial degree $p$", ylabel="RMSE", xlim=(0.5, p.max() + 0.5))
587     ax.legend(fontsize=7.5, loc="lower left")
588     ax.text(chosen + 0.1, 0.97, f"selected $p$={chosen}", transform=ax.get_xaxis_transform(),
589            va="top", fontsize=7.5, color="0.25")
590     if len(under):
591         ax.text(0.5 * (under.min() + p.max()), 1.01, "terms > points\nper CV fold",
592                transform=ax.get_xaxis_transform(), fontsize=7, color="0.35",
593                ha="center", va="bottom")
594     panel_letter(ax, "a")
595     axc.semilogy(p, sweep["cond_design"], "o-", color="0.15", ms=4.5)
596     axc.set(xlabel="polynomial degree $p$", ylabel=r"cond$(\mathbf{\Psi})$", xlim=(0.5, p.max() + 0.5))
597     panel_letter(axc, "b")
598     fig.tight_layout()
599     save_figure(fig, out, "fig04_degree_sweep")
600
601 def plot_parity(y_test, preds, labels, out):
602     fig, axes = plt.subplots(1, 3, figsize=(7.0, 2.55))
603     lo = min(y_test.min(), min(v.min() for v in preds.values()))
604     hi = max(y_test.max(), max(v.max() for v in preds.values()))
605     pad = 0.05 * (hi - lo)
606     lim = (lo - pad, hi + pad)
607     for ax, name, letter in zip(axes, MODEL_NAMES, "abc"):
608         ax.plot(lim, lim, "-", color="0.5", lw=1.0, zorder=1)
609         ax.plot(y_test, preds[name], MODEL_MARKERS[name], ms=5, mfc=MODEL_COLORS[name],
610                mec="black", mew=0.5, ls="none", zorder=3)
611         m = metrics(y_test, preds[name])
612         ax.text(0.97, 0.04, f"{labels[name]}\nRMSE $\{latex\_sci(m['RMSE'])\}$\nR^2={m['R2']:.6f}$",
613                transform=ax.transAxes, ha="right", va="bottom", fontsize=7.5,
614                bbox=dict(boxstyle="round,pad=0.25", fc="white", ec="0.7", alpha=0.9))
615         ax.set(xlim=lim, ylim=lim, aspect="equal", xlabel="true $\hat{f}$ (test set)")
616         ax.grid(alpha=0.3, lw=0.5)
617         panel_letter(ax, letter)
618     axes[0].set_ylabel(r"predicted $\hat{f}$")
619     fig.tight_layout()
620     save_figure(fig, out, "fig05_parity")
621
622
623 def plot_side_by_side(spec, G, Y_hat, X_train, label, out, name):
624     X1, X2, Y = G["X1"], G["X2"], G["Y"]
625     E = Y - Y_hat
626     levels, cmap = field_levels(Y)
627     emax = np.abs(E).max()
628     fig, axes = plt.subplots(1, 3, figsize=(7.4, 2.35))
629     panels = [(Y, "True $\hat{f}$", "$\hat{f}$"), (Y_hat, f"{label} $\hat{f}$", "$\hat{f}$")]
630     for ax, (Z, title, cb_label) in zip(axes[:2], panels):
631         # both panels use the same levels, so a colour means the same value;
632         # extend="both" shows (rather than hides) surrogate values outside them
633         cs = ax.contourf(X1, X2, Z, levels=levels, cmap=cmap, extend="both")
634         cs.set_rasterized(True)
635         ax.contour(X1, X2, Z, levels=spec["line_levels"], colors="white", linewidths=0.5, alpha=0.75)
636         ax.set_title(title, fontsize=8.5, pad=3)
637         sci_colorbar(fig, cs, ax, cb_label)
638     axes[0].plot(*spec["minima"].T, "*", ms=10, mfc="#e34948", mec="white", mew=0.7, ls="none")
639     axes[1].plot(X_train[:, 0], X_train[:, 1], "o", ms=2.8, mfc="white", mec="black", mew=0.4, ls="none")
640     cs = axes[2].contourf(X1, X2, E, levels=np.linspace(-emax, emax, 41), cmap="RdBu_r",

```

```

641         vmin=-emax, vmax=emax)
642     cs.set_rasterized(True)
643     axes[2].plot(X_train[:, 0], X_train[:, 1], "o", ms=3, mfc="#eda100", mec="black", mew=0.45,
644                 ls="none")
645     axes[2].set_title("Error  $f - \hat{f}$ ", fontsize=8.5, pad=3)
646     sci_colorbar(fig, cs, axes[2], " $f - \hat{f}$ ")
647     for ax in axes:
648         ax.set(xlabel="$x_1$", ylabel="$x_2$", aspect="equal")
649     fig.tight_layout(w_pad=0.6)
650     save_figure(fig, out, f"fig06_side_by_side_{name}")
651
652
653 def plot_surfaces(spec, G, Y_hat, X_train, y_train, label, out, name):
654     X1, X2, Y = G["X1"], G["X2"], G["Y"]
655     vmin, vmax = Y.min(), Y.max()
656     zlim = (min(vmin, Y_hat.min()), max(vmax, Y_hat.max()))
657     fig = plt.figure(figsize=(7.0, 3.0))
658     for i, (Z, title) in enumerate([(Y, "True  $f$ "), (Y_hat, f"{label}  $\hat{f}$ )]):
659         ax = fig.add_subplot(1, 2, i + 1, projection="3d")
660         surf = ax.plot_surface(X1, X2, Z, cmap="viridis", vmin=vmin, vmax=vmax, linewidth=0,
661                               rstride=2, cstride=2, antialiased=True, rasterized=True)
662         if i == 1:
663             ax.scatter(X_train[:, 0], X_train[:, 1], y_train, c="#e34948", s=7,
664                       depthshade=False, label="training data")
665             ax.legend(loc="upper right", fontsize=7.5)
666             ax.set(xlabel="$x_1$", ylabel="$x_2$", zlabel="$f$ if  $i == 0$  else  $r \hat{f}$ ",
667                   zlim=zlim, xlim=spec["bounds"][0], ylim=spec["bounds"][1])
668             ax.set_title(title, fontsize=8.5, pad=0)
669             ax.view_init("spec["view"]")
670             ax.set_box_aspect(None, zoom=0.86)
671     fig.colorbar(surf, ax=fig.axes, shrink=0.7, pad=0.04, label="$f$,  $\hat{f}$  (shared scale)")
672     save_figure(fig, out, f"fig07_surfaces_{name}")
673
674
675 def plot_uncertainty(G, sigma, abs_err, X_train, out):
676     fig = plt.figure(figsize=(7.4, 2.45))
677     axes = [fig.add_subplot(1, 3, i + 1) for i in range(3)]
678     # both fields span several decades and peak in one corner, so a linear
679     # scale would show a single colour; the log scale keeps the interior visible
680     for ax, Z, label, letter in [(axes[0], sigma, r"GP std. dev.  $\hat{\sigma}$ ", "a"),
681                                 (axes[1], abs_err, r"true  $f - \hat{f}$ ", "b")]:
682         Zc = np.maximum(Z, Z.max() * 1e-5)
683         levels = np.logspace(np.log10(Zc.min()), np.log10(Zc.max()), 41)
684         cs = ax.contourf(G["X1"], G["X2"], Zc, levels=levels, cmap="cividis", norm=LogNorm())
685         cs.set_rasterized(True)
686         ax.plot(X_train[:, 0], X_train[:, 1], "o", ms=2.8, mfc="white", mec="black", mew=0.4, ls="none")
687         ax.set(xlabel="$x_1$", ylabel="$x_2$", aspect="equal")
688         cb = fig.colorbar(cs, ax=ax, pad=0.02, fraction=0.05, label=label)
689         cb.locator = LogLocator()
690         cb.update_ticks()
691         panel_letter(ax, letter)
692     ax = axes[2]
693     s, e = sigma.ravel(), abs_err.ravel()
694     keep = np.random.default_rng(SEED).choice(len(s), SCATTER_POINTS, replace=False)
695     ax.loglog(s[keep], np.maximum(e[keep], 1e-16), ".", ms=2, color="#1baf7a", alpha=0.5)
696     ref = np.logspace(np.log10(s.min()), np.log10(s.max()), 10)
697     ax.loglog(ref, 2 * ref, "--", color="0.2", lw=1.0, label=r" $|e| = 2 \hat{\sigma}$ ")
698     ax.set(xlabel=r" $\hat{\sigma}$ ", ylabel=r" $f - \hat{f}$ ")
699     ax.legend(fontsize=7.5, loc="upper left")
700     ax.set_box_aspect(1)
701     ax.grid(alpha=0.3, lw=0.5, which="major")
702     panel_letter(ax, "c")
703     fig.tight_layout(w_pad=0.5)
704     save_figure(fig, out, "fig08_gp_uncertainty")
705
706
707 def plot_learning_curve(curve, y_grid_std, out):
708     fig, axes = plt.subplots(1, 2, figsize=(7.0, 2.9), sharey=True)
709     for ax, col, ylabel, letter in [(axes[0], "grid_RMSE", "RMSE on the dense grid", "a"),
710                                   (axes[1], "core_RMSE", "", "b")]:
711         for name in MODEL_NAMES:
712             d = curve[curve["model"] == name].groupby("N")[col]
713             med, lo, hi = d.median(), d.min(), d.max()
714             n = med.index.values
715             # band = spread over the repeated LHS draws, line = median
716             ax.fill_between(n, np.maximum(lo, 1e-15), np.maximum(hi, 1e-15),
717                             color=MODEL_COLORS[name], alpha=0.15, lw=0)
718             ax.loglog(n, np.maximum(med, 1e-15), marker=MODEL_MARKERS[name],
719                       color=MODEL_COLORS[name], ms=4.5, label=name)
720             ax.axhline(y_grid_std, color="0.4", ls=":", lw=1.2, label="mean predictor (std of  $f$ )")
721             ax.set(xlabel="training points  $N$ ", ylabel=ylabel)

```

```

722     ax.set_xticks(LEARNING_N)
723     ax.set_xticklabels([str(n) for n in LEARNING_N])
724     ax.xaxis.set_minor_locator(plt.NullLocator())
725     ax.grid(alpha=0.3, lw=0.5)
726     panel_letter(ax, letter)
727     axes[1].set_ylabel(f"RMSE in the central box ({int(CORE_FRAC * 100)}% of each side)")
728     axes[1].yaxis.set_tick_params(labelleft=True)
729     handles, names = axes[0].get_legend_handles_labels()
730     fig.tight_layout(rect=(0, 0.08, 1, 1))
731     fig.legend(handles, names, loc="lower center", ncol=4, fontsize=7.5)
732     save_figure(fig, out, "fig09_learning_curve")
733
734
735 def plot_optimisation(spec, G, surfaces, found, starts, out):
736     levels, cmap = field_levels(G["Y"])
737     fig, axes = plt.subplots(1, 3, figsize=(7.4, 2.55))
738     for ax, name, letter in zip(axes, MODEL_NAMES, "abc"):
739         cs = ax.contourf(G["X1"], G["X2"], surfaces[name], levels=levels, cmap=cmap,
740                         extend="both")
741         cs.set_rasterized(True)
742         ax.contour(G["X1"], G["X2"], surfaces[name], levels=spec["line_levels"], colors="white",
743                  linewidths=0.45, alpha=0.7)
744         ax.plot(starts[:, 0], starts[:, 1], ".", ms=3, color="white", alpha=0.9)
745         xs = np.array([c["x"] for c in found[name]])
746         ax.plot(*spec["minima"].T, "x", ms=13, mfc="#e34948", mec="white", mew=0.8, ls="none")
747         ax.plot(xs[:, 0], xs[:, 1], "x", ms=8, mfc="#eda100", mec="black", mew=0.7, ls="none")
748         ax.set(xlabel="$x_1$", ylabel="$x_2$", if letter == "a" else "", aspect="equal")
749         ax.text(0.03, 0.04, name, transform=ax.transAxes, fontsize=7.5, color="white")
750         panel_letter(ax, letter)
751         handles = [plt.Line2D([], [], marker=".", color="white", mec="0.3", ls="none", ms=6),
752                  plt.Line2D([], [], marker="x", mfc="#e34948", mec="white", ls="none", ms=11),
753                  plt.Line2D([], [], marker="x", mfc="#eda100", mec="black", ls="none", ms=8)]
754         fig.legend(handles, ["start points", "known global minimum", "surrogate minima"],
755                  loc="lower center", ncol=3, fontsize=7.5, bbox_to_anchor=(0.45, -0.07))
756         cb = fig.colorbar(cs, ax=axes, pad=0.015, fraction=0.03, label=r"$\hat{f}$")
757         cb.set_ticks([t for t in MaxNLocator(6).tick_values(levels[0], levels[-1])
758                      if levels[0] <= t <= levels[-1]])
759         save_figure(fig, out, "fig10_optimisation")
760
761
762 def plot_extrapolation(spec, G, Y_hat, X_sub, label, out, name):
763     X1, X2, Y = G["X1"], G["X2"], G["Y"]
764     levels, cmap = field_levels(Y)
765     E = Y - Y_hat
766     emax = np.abs(E).max()
767     sb = spec["sub_box"]
768     fig, axes = plt.subplots(1, 3, figsize=(7.4, 2.35))
769     for ax, Z, title, cb_label in [(axes[0], Y, "True $f$", "$f$"),
770                                   (axes[1], Y_hat, f"{label} $\hat{f}$ (sub-box training)",
771                                    "$\hat{f}$")]:
772         # outside the sub-box the surrogate may leave the true range; extend="both"
773         # shows that instead of rescaling the colours to hide it
774         cs = ax.contourf(X1, X2, Z, levels=levels, cmap=cmap, extend="both")
775         cs.set_rasterized(True)
776         ax.set_title(title, fontsize=8.5, pad=3)
777         sci_colorbar(fig, cs, ax, cb_label)
778     axes[1].plot(X_sub[:, 0], X_sub[:, 1], "o", ms=2.6, mfc="white", mec="black", mew=0.4, ls="none")
779     cs = axes[2].contourf(X1, X2, E, levels=np.linspace(-emax, emax, 41), cmap="RdBu_r")
780     cs.set_rasterized(True)
781     axes[2].set_title("Error $f - \hat{f}$", fontsize=8.5, pad=3)
782     sci_colorbar(fig, cs, axes[2], "$f - \hat{f}$")
783     for ax in axes:
784         ax.add_patch(Rectangle((sb[0, 0], sb[1, 0]), sb[0, 1] - sb[0, 0], sb[1, 1] - sb[1, 0],
785                               fill=False, ec="#e34948", lw=1.6))
786         ax.set(xlabel="$x_1$", ylabel="$x_2$", aspect="equal")
787     fig.tight_layout(w_pad=0.6)
788     save_figure(fig, out, f"fig11_extrapolation_{name}")
789
790
791 # ----- the study -----
792
793 def fit_gp_counting_failures(model, X, y):
794     """Fit a GP pipeline and count optimiser runs that ended abnormally.
795
796     The warnings are recorded rather than silenced, so the report can state
797     how often the marginal-likelihood search failed to converge cleanly.
798     """
799     with warnings.catch_warnings(record=True) as caught:
800         warnings.simplefilter("always", ConvergenceWarning)
801         model.fit(X, y)
802     messages = [str(w.message) for w in caught if isinstance(w.category, ConvergenceWarning)]

```

```

803     return sum("ABNORMAL" in msg for msg in messages)
804
805
806 def run_study(key, spec):
807     print(f"\n{spec['name']} function")
808     f = spec["func"]
809     b = spec["bounds"]
810     fig_dir = OUTPUT_DIR / "figures" / key
811     tab_dir = OUTPUT_DIR / "tables" / key
812     tab_dir.mkdir(parents=True, exist_ok=True)
813     summary = {"function": spec["name"], "bounds": b.tolist()}
814
815     def f_vec(X):
816         return f(X[:, 0], X[:, 1])
817
818     # step 1: define the truth and check it at the known minimum before fitting
819     x1 = np.linspace(*b[0], GRID_N)
820     x2 = np.linspace(*b[1], GRID_N)
821     X1, X2 = np.meshgrid(x1, x2)
822     G = {"X1": X1, "X2": X2, "Y": f(X1, X2)}
823     X_grid = np.column_stack([X1.ravel(), X2.ravel()])
824     y_grid = G["Y"].ravel()
825     summary["truth"] = {"f_at_known_min": float(f(*spec["minima"])),
826                        "grid_min": float(y_grid.min()), "grid_max": float(y_grid.max()),
827                        "grid_mean": float(y_grid.mean()), "grid_std": float(y_grid.std())}
828     plot_truth(spec, G, fig_dir)
829
830     # step 2: designs compared in the unit square with the same budget
831     rng = np.random.default_rng(SEED)
832     designs = {"Uniform random": uniform_random(N_SAMPLES, 2, rng),
833              "LHS": latin_hypercube(N_SAMPLES, 2, rng),
834              "Maximin LHS": maximin_lhs(N_SAMPLES, 2, rng, MAXIMIN_CANDIDATES)}
835     scores = compare_designs(np.random.default_rng(SEED_DOE_STUDY))
836     summary["doe"] = {k: float(min_distance(U)) for k, U in designs.items()}
837     summary["doe_realisations"] = {k: {"median": float(np.median(s)), "mean": float(s.mean()),
838                                       "p05": float(np.percentile(s, 5)), "max": float(s.max())}
839                                   for k, s in scores.items()}
840     plot_doe(spec, designs, scores, summary["doe"]["Maximin LHS"], fig_dir)
841
842     X_doe = to_box(designs["Maximin LHS"], b)
843     y_doe = f_vec(X_doe) # the N expensive evaluations
844
845     # step 3: the test rows are set aside here and only used after all choices are fixed
846     X_train, X_test, y_train, y_test = train_test_split(
847         X_doe, y_doe, test_size=TEST_FRACTION, random_state=SEED)
848     split = pd.concat([pd.DataFrame({"set": "train", "x1": X_train[:, 0], "x2": X_train[:, 1], "f": y_train}),
849                      pd.DataFrame({"set": "test", "x1": X_test[:, 0], "x2": X_test[:, 1], "f": y_test})])
850     split["dist_to_x_min"] = np.min(np.linalg.norm(
851         split[["x1", "x2"]].to_numpy()[:, None, :] - spec["minima"][None, :, :], axis=2), axis=1)
852     split.to_csv(tab_dir / "doe_split.csv", index=False)
853     summary["split"] = {"n_train": len(X_train), "n_test": len(X_test),
854                       "y_test_range": float(y_test.max() - y_test.min()),
855                       "corner_gaps": corner_gaps(X_train, b)}
856     plot_split(spec, G, X_train, X_test, fig_dir)
857
858     # steps 4-5: polynomial degree chosen by training-only CV, then locked;
859     # the test and grid curves are attached afterwards purely as diagnostics
860     sweep = degree_sweep(X_train, y_train)
861     degree = select_degree(sweep, y_train)
862     sweep = add_sweep_diagnostics(sweep, X_train, y_train, X_test, y_test, X_grid, y_grid)
863     sweep.to_csv(tab_dir / "degree_sweep.csv", index=False)
864     summary["selected_degree"] = degree
865     plot_degree_sweep(sweep, degree, fig_dir)
866
867     models = {name: make_model(name, degree) for name in MODEL_NAMES}
868     models["Polynomial"].fit(X_train, y_train)
869     models["Cubic RBF"].fit(X_train, y_train)
870     n_abnormal = fit_gp_counting_failures(models["Gaussian process"], X_train, y_train)
871     labels = {"Polynomial": f"Polynomial ($p={degree}$)", "Cubic RBF": "Cubic RBF",
872             "Gaussian process": "GP"}
873
874     readback = polynomial_readback(models["Polynomial"], degree, X_grid)
875     exact = spec["exact_coefs"]
876     coef_rows = [{"term": t, "fitted": c, "exact": exact.get(t, 0.0),
877                  "abs_diff": abs(c - exact.get(t, 0.0))} for t, c in readback.items()]
878     pd.DataFrame(coef_rows).to_csv(tab_dir / "polynomial_coefficients.csv", index=False)
879     summary["coef_max_abs_diff"] = float(max(r["abs_diff"] for r in coef_rows))
880     summary["rbf_condition_number"] = float(models["Cubic RBF"][-1].condition_number_)
881     summary["gp"] = gp_hyperparameters(models["Gaussian process"])
882     summary["gp"]["optimizer_runs"] = GP_RESTARTS + 1
883     summary["gp"]["abnormal_terminations"] = n_abnormal

```

```

884 if summary["gp"]["parameters_at_bounds"]:
885     gp_bound_sensitivity(X_train, y_train).to_csv(tab_dir / "gp_bound_sensitivity.csv",
886                                           index=False)
887
888 if key == "trid":
889     # hand derivation: grad f = 0 -> [2 -1; -1 2] x = [2; 2]
890     H_true = np.array([[2.0, -1.0], [-1.0, 2.0]])
891     x_hand = np.linalg.solve(H_true, np.array([2.0, 2.0]))
892     x_fit, H_fit = quadratic_stationary_point(readback)
893     summary["hand_check"] = {"x_hand": x_hand.tolist(), "f_hand": float(f(*x_hand)),
894                             "hessian_eigenvalues": np.linalg.eigvalsh(H_true).tolist(),
895                             "x_from_fitted_quadratic": x_fit.tolist(),
896                             "fitted_hessian": H_fit.tolist()}
897
898 # step 6: held-out metrics against the mean-of-training baseline
899 preds = {name: m.predict(X_test) for name, m in models.items()}
900 rows = [{"model": "Baseline mean(y_train)",
901         **metrics(y_test, np.full_like(y_test, y_train.mean()))}]
902 rows += [{"model": labels[n].replace("$", ""), **metrics(y_test, preds[n])} for n in MODEL_NAMES]
903 test_table = pd.DataFrame(rows)
904 test_table.to_csv(tab_dir / "test_metrics.csv", index=False)
905 plot_parity(y_test, preds, labels, fig_dir)
906
907 # step 7: CV on the training partition, with a dense-grid diagnostic next to it
908 grid_pred = {n: models[n].predict(X_grid) for n in ("Polynomial", "Cubic RBF")}
909 grid_pred["Gaussian process"], gp_std = models["Gaussian process"].predict(X_grid, return_std=True)
910 cv_rows = [{"model": labels[n].replace("$", ""),
911            "CV_RMSE": cv_rmse(make_model(n, degree), X_train, y_train),
912            "test_RMSE": metrics(y_test, preds[n])["RMSE"],
913            **{f"grid_{k}": v for k, v in metrics(y_grid, grid_pred[n]).items()}}
914         for n in MODEL_NAMES]
915 cv_table = pd.DataFrame(cv_rows)
916 cv_table.to_csv(tab_dir / "cv_and_grid.csv", index=False)
917 summary["best_by_cv"] = MODEL_NAMES[int(np.argmax(cv_table["CV_RMSE"]))]
918
919 # step 8: side-by-side maps for every model. The CV-best polynomial is the
920 # required figure, but its error map is pure round-off here, so the RBF and
921 # GP maps are the ones that show where a model without the true form fails
922 surfaces = {n: grid_pred[n].reshape(X1.shape) for n in MODEL_NAMES}
923 tags = {"Polynomial": "poly", "Cubic RBF": "rbf", "Gaussian process": "gp"}
924 for n in MODEL_NAMES:
925     plot_side_by_side(spec, G, surfaces[n], X_train, labels[n], fig_dir, tags[n])
926     plot_surfaces(spec, G, surfaces[n], X_train, y_train, labels[n], fig_dir, tags[n])
927
928 # the core region drops the outer band, to separate edge effects from the
929 # interior; it is used for the uncertainty floor and the learning curve
930 centre, half = b.mean(axis=1), 0.5 * CORE_FRAC * (b[:, 1] - b[:, 0])
931 core = np.all(np.abs(X_grid - centre) <= half, axis=1)
932
933 # step 9: GP standard deviation against the error it cannot see
934 abs_err = np.abs(G["Y"] - surfaces["Gaussian process"])
935 sigma = gp_std.reshape(X1.shape)
936 outside = abs_err > 2 * sigma
937 # jitter plus noise put a floor under sigma_hat; where sigma_hat sits on it,
938 # it carries no information about where the error is
939 gp = models["Gaussian process"][-1]
940 sigma_floor = np.sqrt(gp.alpha + gp.kernel_.k2.noise_level) * y_train.std()
941 summary["gp_uncertainty"] = {
942     "pearson": float(np.corrcoef(sigma.ravel(), abs_err.ravel())[0, 1]),
943     "spearman": float(spearmanr(sigma.ravel(), abs_err.ravel())[0]),
944     "coverage_2sigma": float(np.mean(~outside)),
945     "sigma_max": float(sigma.max()), "abs_err_max": float(abs_err.max()),
946     # where the 2-sigma band fails matters more than how often
947     "abs_err_max_outside_band": float(abs_err[outside].max()) if outside.any() else 0.0,
948     "centroid_outside_band": ([float(X1[outside].mean()), float(X2[outside].mean())]
949                             if outside.any() else None),
950     "sigma_floor": float(sigma_floor),
951     "sigma_at_train_max": float(models["Gaussian process"].predict(
952         X_train, return_std=True)[1].max()),
953     "sigma_median_core": float(np.median(gp_std[core]))}
954 plot_uncertainty(G, sigma, abs_err, X_train, fig_dir)
955
956 # step 10: learning curve on fixed grid, several LHS draws per size
957 lc_rows = []
958 for n in LEARNING_N:
959     p = degree
960     while n_terms(p) > n and p > 1: # keep the least-squares problem determined
961         p -= 1
962     for r in range(LEARNING_REPEATS):
963         U = latin_hypercube(n, 2, np.random.default_rng(SEED + SEED_STRIDE * r + n))
964         Xn = to_box(U, b)

```

```

965     yn = f_vec(Xn)
966     for name in MODEL_NAMES:
967         with warnings.catch_warnings():
968             warnings.simplefilter("ignore", ConvergenceWarning)
969             m = make_model(name, p).fit(Xn, yn)
970             yg = m.predict(X_grid)
971             lc_rows.append({"N": n, "repeat": r, "model": name, "degree_used": p,
972                           "grid_RMSE": metrics(y_grid, yg)["RMSE"],
973                           "core_RMSE": metrics(y_grid[core], yg[core])["RMSE"],
974                           "rbf_condition_number": (m[-1].condition_number_
975                                                    if name == "Cubic RBF" else np.nan)})
976
977 curve = pd.DataFrame(lc_rows)
978 curve.to_csv(tab_dir / "learning_curve.csv", index=False)
979 lc_summary = (curve.groupby(["model", "N"])[["grid_RMSE", "core_RMSE", "rbf_condition_number"]]
980              .agg(["median", "min", "max"]))
981 lc_summary.columns = ["_".join(c) for c in lc_summary.columns]
982 lc_summary.reset_index().to_csv(tab_dir / "learning_curve_summary.csv", index=False)
983 summary["core_area_fraction"] = CORE_FRAC ** 2
984 plot_learning_curve(curve, y_grid.std(), fig_dir)
985
986 # step 11: optimise every surrogate, then pay one true evaluation per candidate
987 cluster_tol = CLUSTER_FRAC * (b[0, 1] - b[0, 0])
988 starts = to_box(latin_hypercube(N_STARTS, 2, np.random.default_rng(SEED_STARTS)), b)
989 found, opt_rows, n_verify = {}, [], 0
990 for name in MODEL_NAMES:
991     clusters, _ = multistart_minimise(models[name].predict, b, starts, cluster_tol)
992     found[name] = clusters
993     for rank, c in enumerate(clusters, start=1):
994         f_true = float(f(*c["x"]))
995         n_verify += 1
996         opt_rows.append({
997             "model": name, "rank": rank, "hits": c["hits"],
998             "x1": c["x"][0], "x2": c["x"][1],
999             "f_hat": c["f_hat"], "f_true": f_true,
1000             "abs_pred_error": abs(c["f_hat"] - f_true),
1001             "gap_to_f_min": f_true - spec["f_min"],
1002             "dist_to_x_min": float(np.min(np.linalg.norm(spec["minima"] - c["x"], axis=1))),
1003             "on_boundary": on_boundary(c["x"], b)})
1004 true_clusters, true_nfev = multistart_minimise(f_vec, b, starts, cluster_tol)
1005 for rank, c in enumerate(true_clusters, start=1):
1006     opt_rows.append({
1007         "model": "True function (check)", "rank": rank, "hits": c["hits"],
1008         "x1": c["x"][0], "x2": c["x"][1], "f_hat": np.nan, "f_true": c["f_hat"],
1009         "abs_pred_error": np.nan, "gap_to_f_min": c["f_hat"] - spec["f_min"],
1010         "dist_to_x_min": float(np.min(np.linalg.norm(spec["minima"] - c["x"], axis=1))),
1011         "on_boundary": on_boundary(c["x"], b)})
1012 opt_table = pd.DataFrame(opt_rows)
1013 opt_table.to_csv(tab_dir / "optimisation.csv", index=False)
1014 summary["optimisation"] = {"n_starts": N_STARTS, "cluster_tol": cluster_tol,
1015                           "true_function_nfev": int(true_nfev)}
1016 plot_optimisation(spec, G, surfaces, found, starts, fig_dir)
1017
1018 # step 12: train on a reduced box, then look inside and outside it separately
1019 sb = spec["sub_box"]
1020 X_sub = to_box(latin_hypercube(len(X_train), 2, np.random.default_rng(SEED_SUB_BOX)), sb)
1021 y_sub = f_vec(X_sub)
1022 inside = np.all((X_grid >= sb[:, 0]) & (X_grid <= sb[:, 1]), axis=1)
1023 ex_rows = []
1024 for name in MODEL_NAMES:
1025     with warnings.catch_warnings():
1026         warnings.simplefilter("ignore", ConvergenceWarning)
1027         m = make_model(name, degree).fit(X_sub, y_sub)
1028         yg = m.predict(X_grid)
1029         m_in, m_out = metrics(y_grid[inside], yg[inside]), metrics(y_grid[~inside], yg[~inside])
1030         ex_rows.append({"model": name, "RMSE_inside": m_in["RMSE"], "RMSE_outside": m_out["RMSE"],
1031                       "MaxAE_inside": m_in["MaxAE"], "MaxAE_outside": m_out["MaxAE"],
1032                       "RMSE_ratio_out_in": m_out["RMSE"] / m_in["RMSE"]})
1033     if name != "Polynomial":
1034         plot_extrapolation(spec, G, yg.reshape(X1.shape), X_sub, labels[name], fig_dir, tags[name])
1035 pd.DataFrame(ex_rows).to_csv(tab_dir / "extrapolation.csv", index=False)
1036
1037 # budget kept apart from the extra evaluations used only for teaching checks
1038 summary["evaluations"] = {
1039     "main_doe": N_SAMPLES,
1040     "dense_grid": GRID_N**2,
1041     "optimum_verification": n_verify,
1042     "true_function_multistart": int(true_nfev),
1043     "learning_curve": int(sum(LEARNING_N) * LEARNING_REPEATS),
1044     "reduced_box": len(X_sub)}
1045
1046 # a short console summary; the full tables are in the CSV files

```

```

1046 print(f" selected polynomial degree : {degree} (training-only {CV_FOLDS}-fold CV)")
1047 for _, row in test_table.iterrows():
1048     print(f" test {row['model']:<24s}: RMSE = {row['RMSE']:.3e}, NRMSE = {row['NRMSE']:.3e}")
1049 for name in MODEL_NAMES:
1050     best = found[name][0]
1051     print(f" {name:<18s} optimum at ({best['x'][0]:.4f}, {best['x'][1]:.4f}), "
1052           f"true f = {f(*best['x']):.6g}")
1053 return summary
1054
1055
1056 def environment_info():
1057     return {"python": platform.python_version(), "numpy": np.__version__, "scipy": scipy.__version__,
1058            "scikit-learn": sklearn.__version__, "matplotlib": matplotlib.__version__,
1059            "pandas": pd.__version__, "platform": platform.platform(), "seed": SEED}
1060
1061
1062 def main():
1063     set_plot_style()
1064     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
1065     env = environment_info()
1066     print("Environment:", env)
1067     results = {"environment": env}
1068     for key, spec in FUNCTIONS.items():
1069         results[key] = run_study(key, spec)
1070     with open(OUTPUT_DIR / "summary.json", "w") as fh:
1071         json.dump(results, fh, indent=2, default=float)
1072     print(f"\nAll figures and tables written to {OUTPUT_DIR}")
1073
1074
1075 if __name__ == "__main__":
1076     main()

```